# Documentação de Projeto

## para o sistema

# MontaControl

**Versão 1.7**

Projeto de sistema elaborado pelo(s) aluno(s) David Dias Pinto e Vinicius Salles de Oliveira e apresentado ao curso de **Engenharia de Software** da **PUC Minas** como parte do Trabalho de Conclusão de Curso (TCC) sob orientação de conteúdo do professor João Pedro Oliveira Batisteli, orientação acadêmica do professor Leonardo Vilela Cardoso e orientação de TCC II do professor (a ser definido no próximo semestre).

**28/06/2026**

# Tabela de Conteúdo

**Tabela de Conteúdo**
**Histórico de Revisões**
**1. Introdução** **1**
**2. Modelos de Usuário e Requisitos** **1**
2.1 Descrição de Atores 1
2.2 Modelos de Usuários 2
2.3 Modelo de Casos de Uso e Histórias de Usuários 4
**3. Modelos de Projeto** **7**
3.1 Diagrama de Classes 7
3.2 Diagramas de Sequência 9
3.3 Arquitetura 13
3.4 Diagrama de Componentes e Implantação 19
3.5 Diagrama de Estados 21
**4. Projeto de Interface com Usuário** **24**
4.1 Esboço das Interfaces Comuns a Todos os Atores 24
4.2 Esboço das Interfaces Usadas pelo Ator Montador 27
4.3 Esboço das Interfaces Usadas pelo Ator Operador Logístico 33
4.4 Esboço das Interfaces Usadas pelo Ator RH 34
4.5 Esboço das Interfaces Usadas pelo Ator Gestor 39
**5. Glossário e Modelos de Dados** **43**
**6. Casos de Teste** **48**
6.1 Testes de Aceitação 48
6.2 Testes de Integração 61
**7. Cronograma e Processo de Implementação** **67**
7.1 Cronograma 67
7.2 Processo de Implementação 72
**Referências** **73**

# Histórico de Revisões

| Nome | Data | Razões para Mudança | Versão |
|---|---|---|---|
| Criação | 13/04/2026 | Início do documento, definindo seções 2.1, 2.2, 2.3, 3.3, 3.4 e 4.1 | 1.0 |
| Atualização Diagramas | 17/05/2026 | Atualização na estrutura do projeto e definição das seções 3.1 | 1.1 |
| Descrição Diagramas | 19/05/2026 | Descrição dos Diagramas, adição do DER e Interfaces, definição das seções 3.2, 3.5, 4.2, 4.3, 5 | 1.2 |

| | | | |
|---|---|---|---|
| Atualização das informações | 03/06/2026 | Descrição das telas, criação do glossário do banco de dados, criação dos casos de teste e introdução da seção 7. | 1.3 |
| Revisão da escrita | 22/06/2026 | Textos revisados para seguir padrão correto de escrita e formalidade | 1.4 |
| Atualização das informações | 23/06/2026 | Atualização das seções 1, 3.4 e 7 | 1.5 |
| Atualização dos testes | 27/06/2026 | Testes principais mantidos e melhor descritos, seção 6 melhor estruturada | 1.6 |
| Atualização das descrições dos diagramas | 28/06/2026 | Revisão e melhoria das descrições de todos diagramas | 1.7 |

# 1. Introdução

Este documento agrega: 1) a elaboração e revisão de modelos de domínio; 2) e modelos de projeto para o sistema MontaControl. O MontaControl é um sistema integrado de gestão de estoque e controle operacional voltado para a montagem de móveis. A necessidade de modernização tecnológica nesse setor pode ser evidenciada ao observarmos o comportamento de um índice de mercado, como o volume de vendas do comércio varejista. Segundo a Pesquisa Mensal de Comércio (PMC) do IBGE, o varejo de móveis no Brasil acumulou um crescimento de 5,9% no volume de vendas ao longo de 2024, demonstrando um forte aquecimento na demanda que exige operações logísticas altamente escaláveis. A referência principal para a descrição geral do problema, domínio e requisitos do sistema é o documento de especificação que descreve a visão de domínio do sistema. Tal especificação acompanha este documento. Anexo a este documento também se encontra o Glossário.

A proposta de construção da plataforma visa solucionar as frequentes frustrações da operação logística e de campo, substituindo processos manuais, como folhas de ponto em papel e controle visual de estoque, por ferramentas digitais integradas. O fluxo de funcionamento inicia-se no depósito, onde o Operador Logístico utiliza a aplicação web para registrar e efetuar a triagem de peças via leitura de código de barras, agrupando-as automaticamente por pedido e ambiente. Em campo, o Montador atua via aplicativo móvel offline-first, realizando o registro de jornada por geolocalização (GPS), a conferência das cargas, o reporte de avarias com fotos e o envio de atestados médicos, garantindo o funcionamento contínuo mesmo sem conexão com a internet.

Na camada de gestão e administração, o sistema centraliza o monitoramento e resguarda a empresa contra problemas operacionais e passivos trabalhistas. O Gestor da Franquia dispõe de um *dashboard* em tempo real para acompanhar a produtividade da equipe, receber alertas imediatos de avarias para solicitação de reposição e gerenciar o ciclo de empréstimo de ferramentas e validação de EPIs. Em paralelo, o setor de Recursos Humanos utiliza a plataforma para controlar o banco de horas e validar os atestados enviados, cujos dados e arquivos sensíveis são armazenados com segurança no Supabase Storage em total conformidade com a Lei Geral de Proteção de Dados (LGPD.

# 2. Modelos de Usuário e Requisitos

Esta seção apresenta os modelos de usuário e os requisitos do sistema. A apresentação é composta pelas seguintes divisões: descrição dos atores (Seção 2.1); descrição dos modelos de usuário (Seção 2.2); e descrição dos casos de uso e histórias do usuário (Seção 2.3).

## 2.1 Descrição de Atores

Nesta subseção, apresentam-se as descrições de cada um dos atores que interagem com o sistema proposto. A definição dos atores é fundamental para delimitar as responsabilidades e o escopo de acesso de cada perfil dentro da plataforma. Os atores identificados são:

Montador de Móveis: Ator primário responsável pela execução das tarefas em campo. Utiliza o aplicativo móvel para realizar o registro de sua jornada de trabalho (com as 4 marcações diárias conforme a CLT), conferência de materiais via leitura de código de barras, reporte de avarias com evidências fotográficas, envio de atestados médicos e consulta às normas da empresa.

Operador Logístico: Ator responsável pela triagem e organização das peças recebidas da fábrica. Sua interação com o sistema, por meio da aplicação web, foca no cadastro das peças via leitura de código de barras e no agrupamento dos itens por número de pedido e ambiente de montagem, garantindo a correta separação antes do envio ao cliente.

Gestor da Franquia: Ator com perfil de supervisão operacional. Interage com o sistema por meio da aplicação web, acompanhando a produtividade da equipe em tempo real através do dashboard gerencial, recebendo alertas de avarias, solicitando reposição de peças à fábrica e gerenciando a cautela de ferramentas e a validade dos EPIs.

Profissional de Recursos Humanos (RH): Ator administrativo focado na gestão de pessoal. É responsável pelo cadastro de montadores no sistema, gerenciamento da jornada de trabalho (banco de horas, pausas e compensações), análise e validação de atestados médicos, exportação de relatórios de ponto (AFD) e visualização do espelho de ponto para acompanhamento das marcações da equipe.

## 2.2 Modelos de Usuários

Nesta seção são apresentados os modelos de usuários, os quais são descritos em um breve resumo com objetivos, tarefas, habilidades e frustrações. Na Tabela 1 é apresentado o primeiro usuário, que representa os Montadores. Na Tabela 2 é apresentado o usuário que representa o Operador Logístico, responsável pela triagem e organização das peças no depósito. Já na Tabela 3 é apresentado o usuário Gestor da Franquia.

| Bruno | |
|---|---|
| Descrição | 32 anos, montador há 10 anos. É prático e valoriza agilidade, mas sofre com a burocracia do dia a dia no papel. |
| Objetivo | Ter sua jornada contabilizada de forma justa e realizar montagens sem interrupções por falta de material. |
| Habilidades | Experiência técnica em montagem de móveis; comunicação interpessoal; leitura de projetos impressos. |
| Tarefas | Preencher folhas de ponto manuais em papel; conferir caixas visualmente antes de carregar; ligar para a central para relatar peças quebradas. |

| | |
|---|---|
| Frustrações | Perda ou rasura da folha de ponto; chegar à obra e descobrir que a peça está trocada; dificuldade em comprovar horas extras. |

Tabela 1. Modelo de Usuário: Persona Bruno (Ator Montador).

| Sergio | |
|---|---|
| Descrição | 45 anos, experiente em estoque. É metódico, mas sente dificuldade em manter o controle apenas com pranchetas. |
| Objetivo | Organizar o fluxo de saída da fábrica com o mínimo de erros e garantir que o montador leve os itens corretos. |
| Habilidades | Conhecimento prático de logística; liderança operacional; organização espacial de galpões. |
| Tarefas | Conferir entrada e saída de materiais usando listas impressas; agrupar caixas por pedido visualmente; coordenar o carregamento dos veículos. |
| Frustrações | Ilegibilidade de pedidos de papel; desorganização no carregamento; descobrir a falta de itens apenas quando o montador já está no cliente. |

Tabela 2. Modelo de Usuário : Persona Sergio (Ator Operador Logístico).

| Mariana | |
|---|---|
| Descrição | 38 anos, administradora. Foca em produtividade, mas gasta muito tempo resolvendo falhas de comunicação operacional. |
| Objetivo | Aumentar a produtividade da equipe, reduzir o tempo de espera do cliente e fechar a folha de pagamento sem erros. |
| Habilidades | Gestão de negócios; análise de planilhas financeiras; resolução de conflitos com clientes. |
| Tarefas | Ligar para montadores para saber o *status* das obras; calcular horas de trabalho via controle de papel; solicitar reposições por e-mail ou telefone. |
| Frustrações | Falta de visibilidade sobre o andamento em campo; lentidão no repasse de avarias; excesso de trabalho manual para fechar a folha do mês. |

Tabela 3. Modelo de Usuário: Persona Mariana (Ator Gestor da Franquia).

## 2.3 Modelo de Casos de Uso e Histórias de Usuários

Esta subseção descreve os casos de uso e as histórias de usuário que contemplam as ações dos usuários no sistema proposto. O objetivo desta etapa é documentar o entendimento dos requisitos e estabelecer o escopo de atuação da plataforma.

### 2.3.1 Diagrama de Casos de Uso

A Figura 1 ilustra o Diagrama de Casos de Uso do sistema MontaControl, identificando os quatro atores primários e os dezenove casos de uso que estruturam as funcionalidades da plataforma. O Montador interage exclusivamente pelo aplicativo mobile, com casos de uso voltados ao registro de jornada, conferência de peças e reporte de avarias. O Operador de Depósito opera pela aplicação *web* para realizar a triagem e o cadastro das peças recebidas da fábrica. O Gestor da Franquia utiliza a aplicação *web* para acompanhamento operacional em tempo real, análise de produtividade e gestão de recursos. O Profissional de RH administra cadastros de colaboradores, validação de atestados médicos e controle do banco de horas. Os relacionamentos <<*include*>> e <<*extend*>> representam, respectivamente, comportamentos obrigatórios e opcionais entre casos de uso. As histórias de usuário correspondentes são detalhadas na Seção 2.3.2.

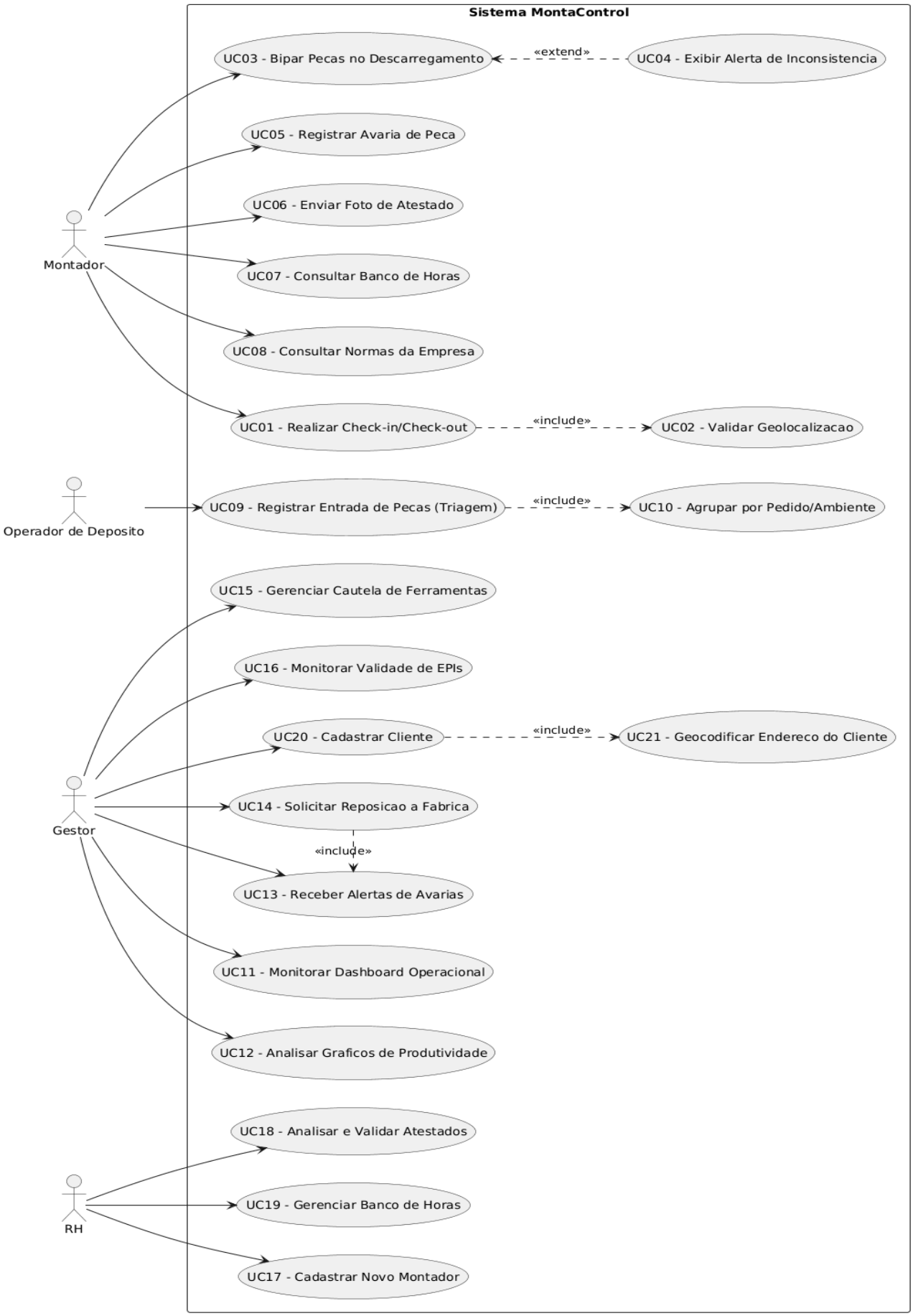

Sistema MontaControl
UC03 - Bipar Pecas no Descarregamento
«extend»
UC04 - Exibir Alerta de Inconsistencia
UC05 - Registrar Avaria de Peca
UC06 - Enviar Foto de Atestado
UC07 - Consultar Banco de Horas
UC08 - Consultar Normas da Empresa
UC01 - Realizar Check-in/Check-out
«include»
UC02 - Validar Geolocalizacao
UC09 - Registrar Entrada de Pecas (Triagem)
«include»
UC10 - Agrupar por Pedido/Ambiente
UC15 - Gerenciar Cautela de Ferramentas
UC16 - Monitorar Validade de EPIs
UC20 - Cadastrar Cliente
«include»
UC21 - Geocodificar Endereco do Cliente
UC14 - Solicitar Reposicao a Fabrica
«include»
UC13 - Receber Alertas de Avarias
UC11 - Monitorar Dashboard Operacional
UC12 - Analisar Graficos de Produtividade
UC18 - Analisar e Validar Atestados
UC19 - Gerenciar Banco de Horas
UC17 - Cadastrar Novo Montador
Montador
Operador de Deposito
Gestor
RH

Figura 1. Diagrama de Casos de Uso do sistema MontaControl.

### 2.3.2 Histórias de Usuário

Nesta subseção, apresentam-se as histórias de usuário (*User Stories*) elicitadas para o sistema. As histórias de usuário consistem em uma forma de representar os requisitos por meio de uma descrição simples, informando quem realizará a ação, qual é essa ação e a razão de sua realização.

As histórias identificadas para o sistema são:

US01. Como montador, desejo registrar meu check-in e check-out via aplicativo com captura de GPS, para que minha jornada de trabalho seja computada corretamente e validada no local da obra (relativo aos UC01 e UC02).

US02. Como montador, desejo ler os códigos de barras das peças no momento do descarregamento, mesmo sem conexão com a internet, para ser alertado caso ocorra erro no pedido ou falta de componentes (relativo aos UC03 e UC04).

US03. Como montador, desejo registrar avarias anexando fotos e categorizando o tipo de problema, para que a central providencie a reposição da peça junto à fábrica (relativo ao UC05).

US04. Como montador, desejo enviar fotos de atestados médicos pelo aplicativo, para que o setor de RH receba e valide a justificativa de ausência (relativo ao UC06).

US05. Como montador, desejo consultar meu saldo de banco de horas e registro de pausas, para ter transparência sobre minha jornada de trabalho (relativo ao UC07).

US06. Como montador, desejo consultar as normas e procedimentos da empresa pelo aplicativo, para seguir os padrões operacionais durante a montagem (relativo ao UC08).

US07. Como operador de depósito, desejo realizar a leitura do código de barras das peças recebidas da fábrica, para que o sistema realize o cadastro e a triagem automática por número de pedido e ambiente de montagem (relativo aos UC09 e UC10).

US08. Como gestor, desejo visualizar um dashboard com alertas em tempo real sobre a operação, para acompanhar a produtividade da equipe de campo (relativo ao UC11).

US09. Como gestor, desejo visualizar gráficos de produtividade comparando metas diárias com o efetivamente registrado, para subsidiar decisões operacionais baseadas em dados (relativo ao UC12).

US10. Como gestor, desejo receber alertas imediatos de avarias reportadas em campo, para solicitar a reposição de peças à fábrica e minimizar o tempo de espera do cliente (relativo aos UC13 e UC14).

US11. Como gestor, desejo registrar o empréstimo de ferramentas e monitorar o vencimento dos Equipamentos de Proteção Individual (EPIs), para evitar perdas patrimoniais e passivos trabalhistas (relativo aos UC15 e UC16).

US12. Como RH, desejo cadastrar novos montadores no sistema, para garantir que apenas profissionais autorizados utilizem a plataforma (relativo ao UC17).

US13. Como RH, desejo analisar e validar atestados médicos enviados pelos montadores, para manter o controle correto de ausências justificadas (relativo ao UC18).

US14. Como RH, desejo gerenciar o banco de horas da equipe, aprovando compensações e visualizando alertas de prazo, para garantir a conformidade com a CLT (relativo ao UC19)

# 3. Modelos de Projeto

Esta seção apresenta os modelos de projeto elaborados para o sistema MontaControl. Os artefatos aqui descritos têm como objetivo representar, de forma estruturada, as decisões de design, a organização dos componentes e o comportamento esperado do sistema. A seção está organizada nas seguintes subseções: o Diagrama de Classes (3.1), que modela as entidades e seus relacionamentos; os Diagramas de Sequência (3.2), que descrevem os fluxos de interação entre atores e subsistemas; a Arquitetura (3.3), apresentada em três níveis do modelo C4; o Diagrama de Componentes e Implantação (3.4), que detalha a estrutura modular e a distribuição física do sistema; e o Diagrama de Estados (3.5), que ilustra os ciclos de vida das principais entidades do domínio.

## 3.1 Diagrama de Classes

A Figura 2 apresenta o diagrama de classes do sistema MontaControl, estruturado em vinte classes distribuídas em sete contextos lógicos: Usuário e Autenticação, Logística e Peças, Avarias, Jornada e Horas, Atestados, Recursos e Normas. A classe Usuário centraliza o controle de acesso baseado em perfis (RBAC), suportando os quatro atores do sistema. O fluxo logístico é modelado pela hierarquia Cliente, Pedido, Ambiente e Peça, em que cada peça pertence a um ambiente por composição. As avarias, atestados e arquivos associados seguem o padrão de entidade com satélite de fotos (AvariaFoto, AtestadoFoto), contendo metadados de armazenamento compatíveis com o Supabase Storage. O controle de jornada reúne as classes Jornada, Pausa e BancoHoras, enquanto a gestão de recursos distingue ferramentas compartilhadas, controladas via Cautela, de EPIs entregues individualmente. Os ciclos de vida das entidades são modelados por enumerados de status e métodos de negócio, como validarLocalizacao(), estaVencido() e estaAtrasada().

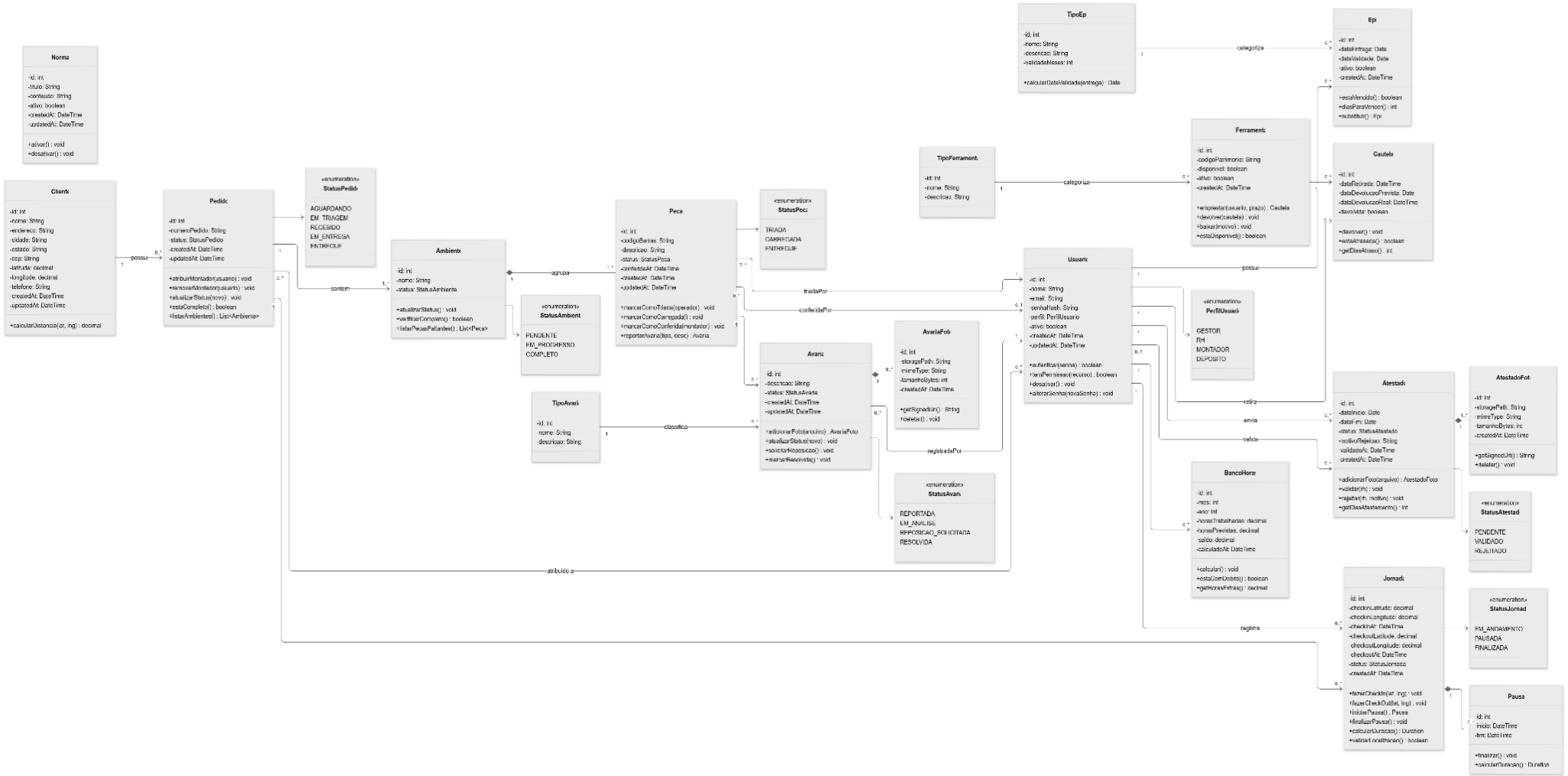



Figura 2. Diagrama de Classes do sistema MontaControl.

## 3.2 Diagramas de Sequência

A Figura 4 ilustra o diagrama de sequência do Ator Gestor da Franquia, cobrindo três frentes de atuação. No monitoramento em tempo real, o navegador mantém uma conexão via WebSocket seguro (WSS) com o servidor para receber notificações instantâneas de *check-ins* e avarias reportadas em campo. Na análise de produtividade, requisições REST consolidam registros de horas e entregas para renderização dos gráficos de meta versus realizado. Na gestão de recursos, o gestor registra cautelas de ferramentas, validade de EPIs e despacha solicitações de reposição de peças à fábrica, persistindo os novos estados no banco de dados.

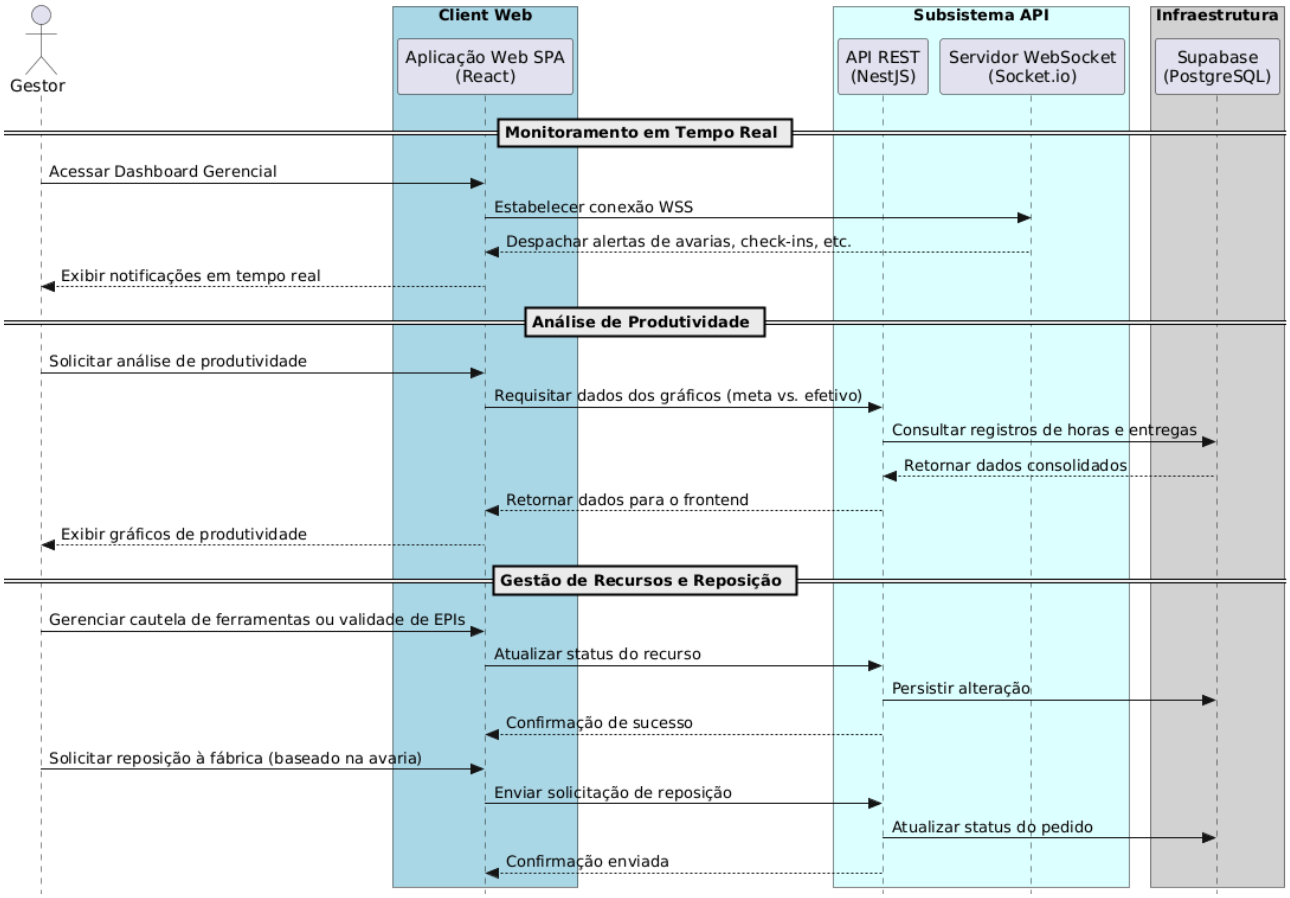



Figura 4. Diagrama de Sequência do Ator Gestor da Franquia.

A Figura 5 apresenta o diagrama de sequência do Ator Operador Logístico, com foco no fluxo de triagem de peças. A leitura do código de barras na interface *web* dispara uma requisição REST à API, que processa o agrupamento automático dos itens por número de pedido e ambiente de

montagem. Os dados são persistidos no PostgreSQL e a confirmação visual retorna ao operador, assegurando a integridade do recebimento da carga.

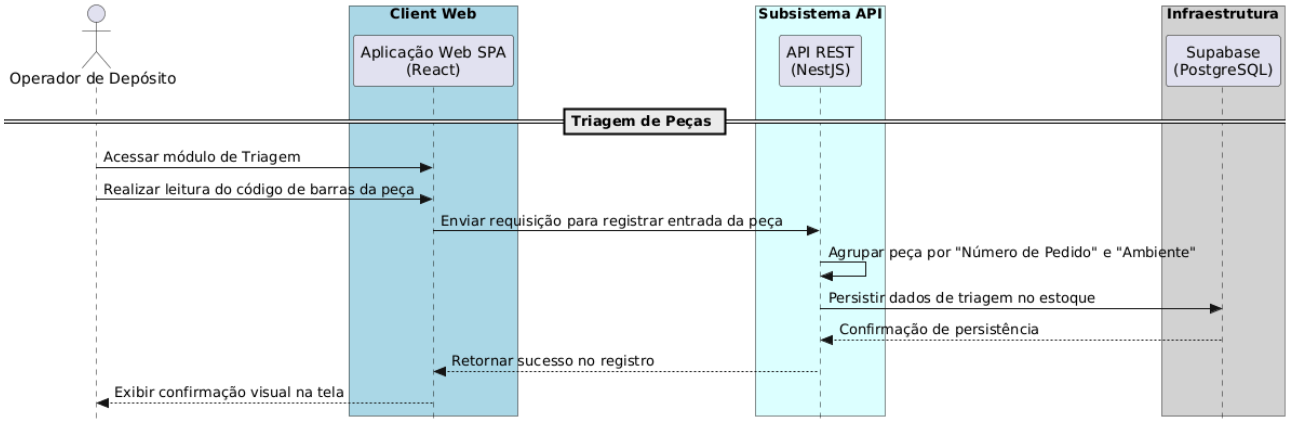



Figura 5. Diagrama de Sequência do Ator Operador Logístico.

A Figura 6 descreve o diagrama de sequência do Ator Montador, evidenciando o comportamento do aplicativo mobile em conjunto com o banco de dados local (WatermelonDB). No recebimento de carga, as peças são validadas localmente contra o pedido ativo, sem necessidade de rede, com alertas imediatos de inconsistência. No registro de jornada, o módulo de GPS captura as coordenadas e as envia via REST para validação e persistência. No registro de avarias e envio de atestados, o aplicativo realiza o *upload* das imagens para o Supabase Storage, sendo que, no caso de avarias, o servidor emite simultaneamente um evento WebSocket para notificar o Gestor em tempo real.

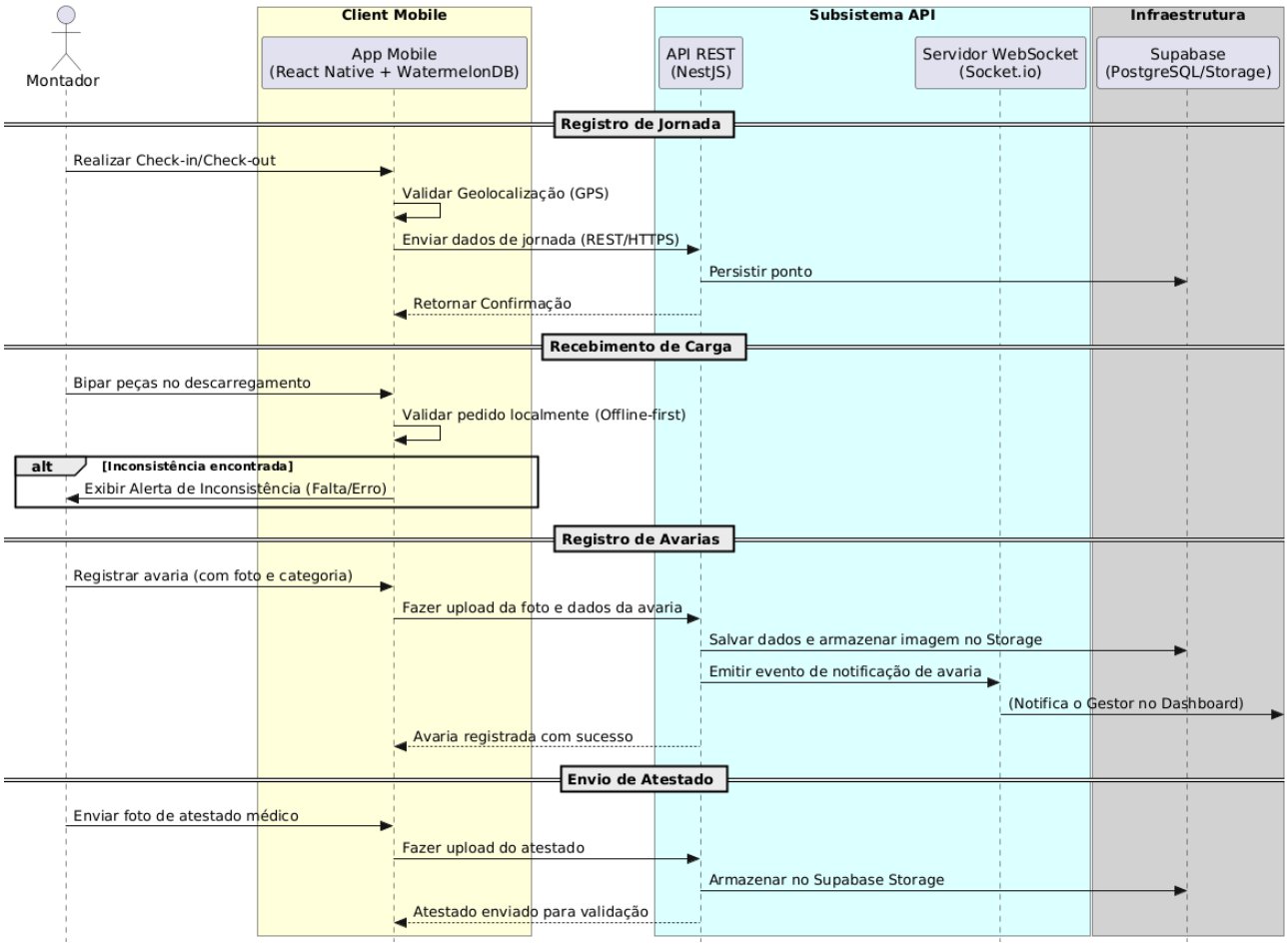



Figura 6. Diagrama de Sequência do Ator Montador.

A Figura 7 detalha o diagrama de sequência do Ator Profissional de Recursos Humanos, abrangendo três fluxos administrativos. Na gestão de cadastros, os dados de um novo montador são submetidos à API REST e persistidos no PostgreSQL. Na validação de atestados, a API recupera referências no banco e consome os arquivos protegidos no Supabase Storage, exibindo o documento ao RH para aprovação ou rejeição com gravação do novo status. Na gestão de jornada, o sistema processa os registros de *check-in*, *check-out* e pausas para compor o espelho de ponto e viabilizar a aprovação de compensações de horas.

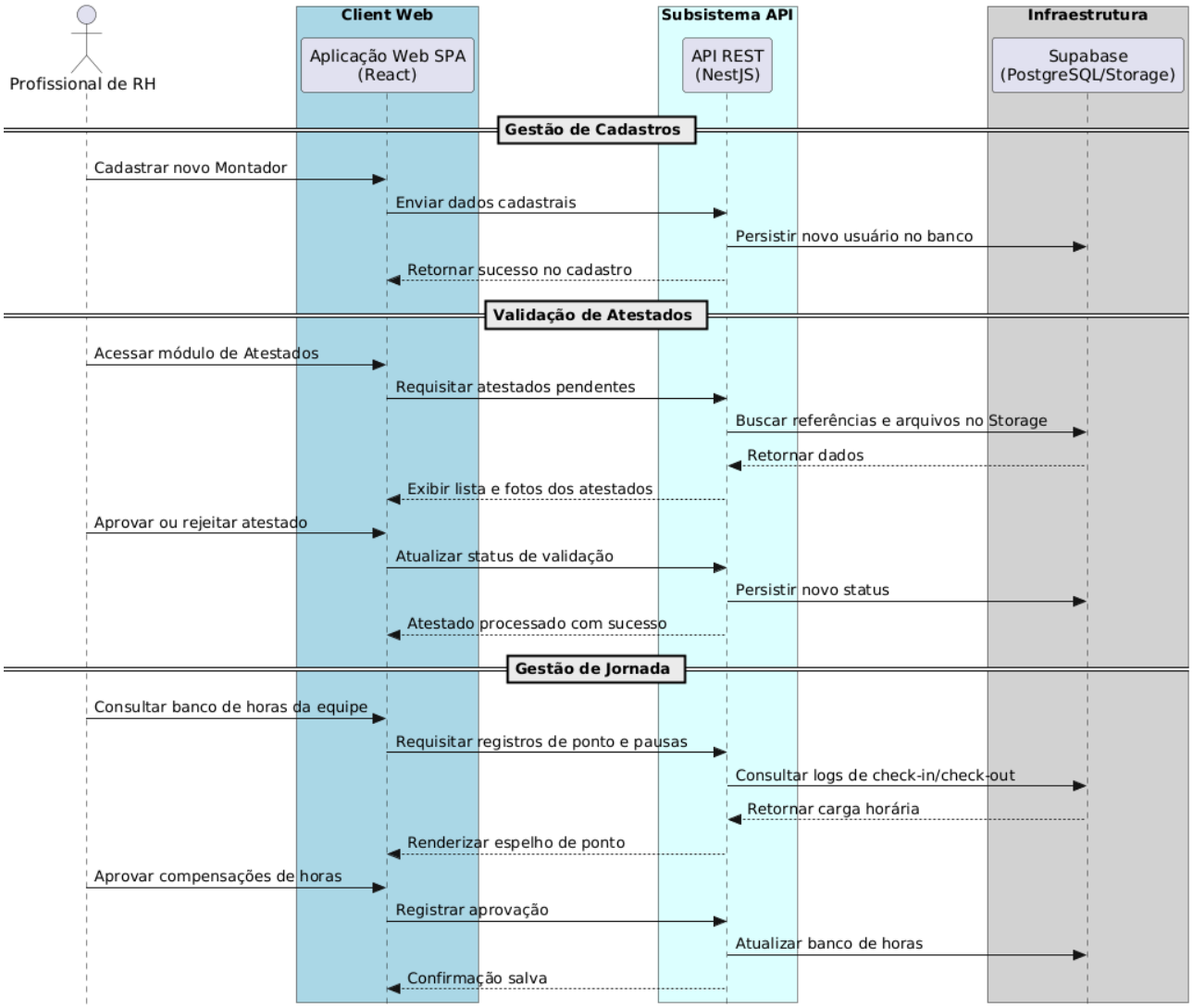



Figura 7. Diagrama de Sequência do Ator Profissional de Recursos Humanos.

## 3.3 Arquitetura

A Figura 8 ilustra o Diagrama de Contexto do sistema, correspondente ao Nível 1 do modelo C4. O MontaControl é identificado como sistema central, interagindo com quatro perfis de usuário: Montador, Gestor, RH e Operador de Depósito. O Montador acessa o sistema por meio do aplicativo mobile com arquitetura *offline-first*, enquanto os demais perfis utilizam a aplicação web. Externamente, o sistema integra-se a uma API de Geolocalização para validação de *check-ins* dentro de um raio de tolerância configurável, e recebe peças etiquetadas provenientes da fábrica, integradas ao fluxo logístico pela leitura do código de barras. A persistência é provida pelo Supabase, que hospeda o PostgreSQL para dados estruturados e o Storage para arquivos sensíveis como fotos de avarias e atestados médicos.

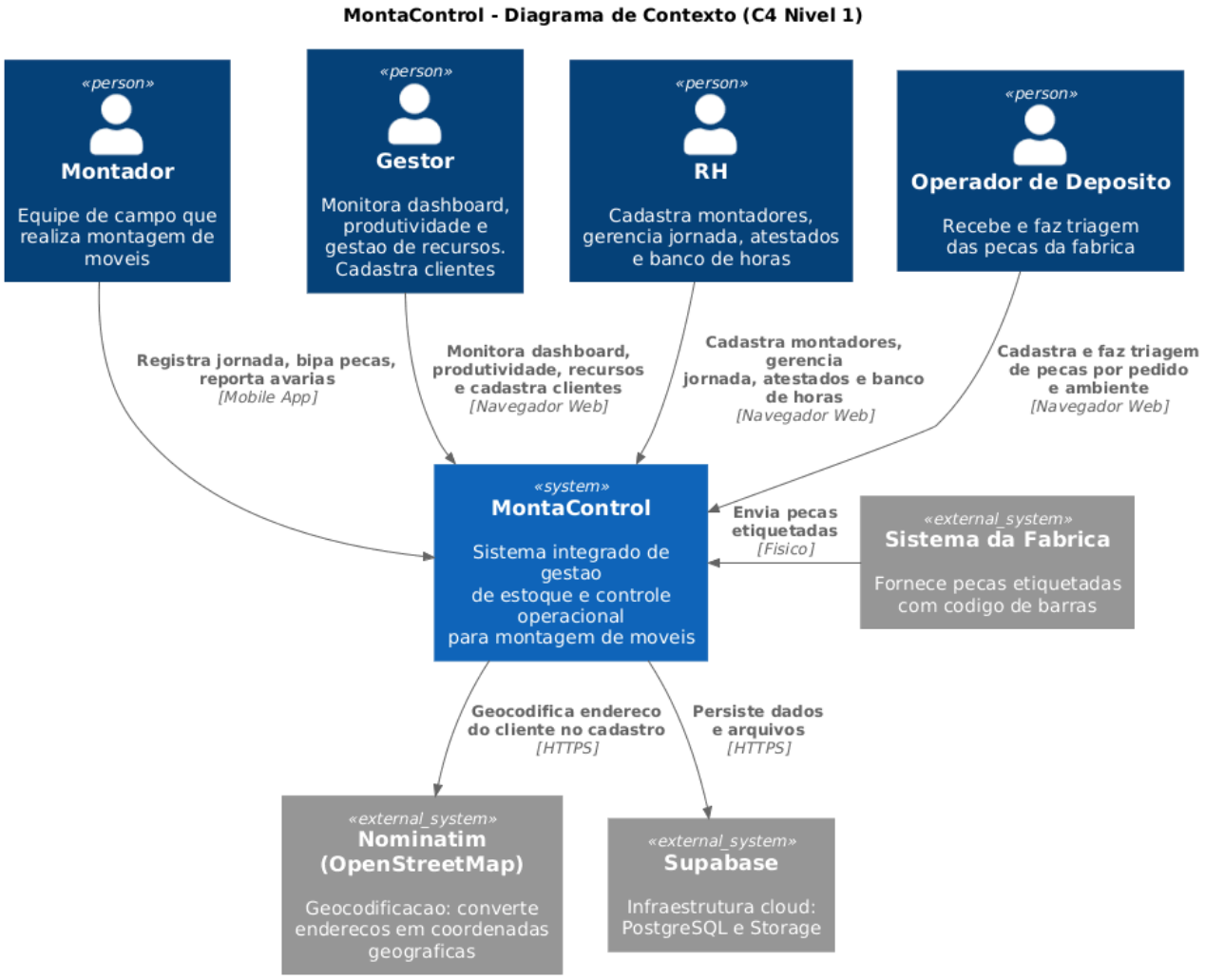



Figura 8. Diagrama de Contexto do Sistema MontaControl (Modelo C4: Nível 1).

A Figura 9 apresenta o Diagrama de *Containers* do sistema (Nível 2 do modelo C4), detalhando os cinco blocos tecnológicos do MontaControl. O App *Mobile*, desenvolvido em React Native com TypeScript, opera com arquitetura *offline-first* e utiliza WatermelonDB para persistência local antes da sincronização. A Aplicação Web SPA, construída em React com TypeScript, serve as interfaces dos perfis administrativos. A API REST, implementada em NestJS, gerencia autenticação JWT, controle de acesso por perfil (RBAC) e acesso a dados via Prisma ORM. O Servidor WebSocket, baseado em Socket.io, é responsável pelo disparo de alertas em tempo real ao Dashboard gerencial. A infraestrutura no Supabase hospeda o PostgreSQL para dados estruturados e o Storage com buckets privados para arquivos sensíveis, acessados via Signed URLs. A comunicação entre *containers* ocorre via REST/HTTPS para operações de CRUD e WebSocket (WSS) para notificações.

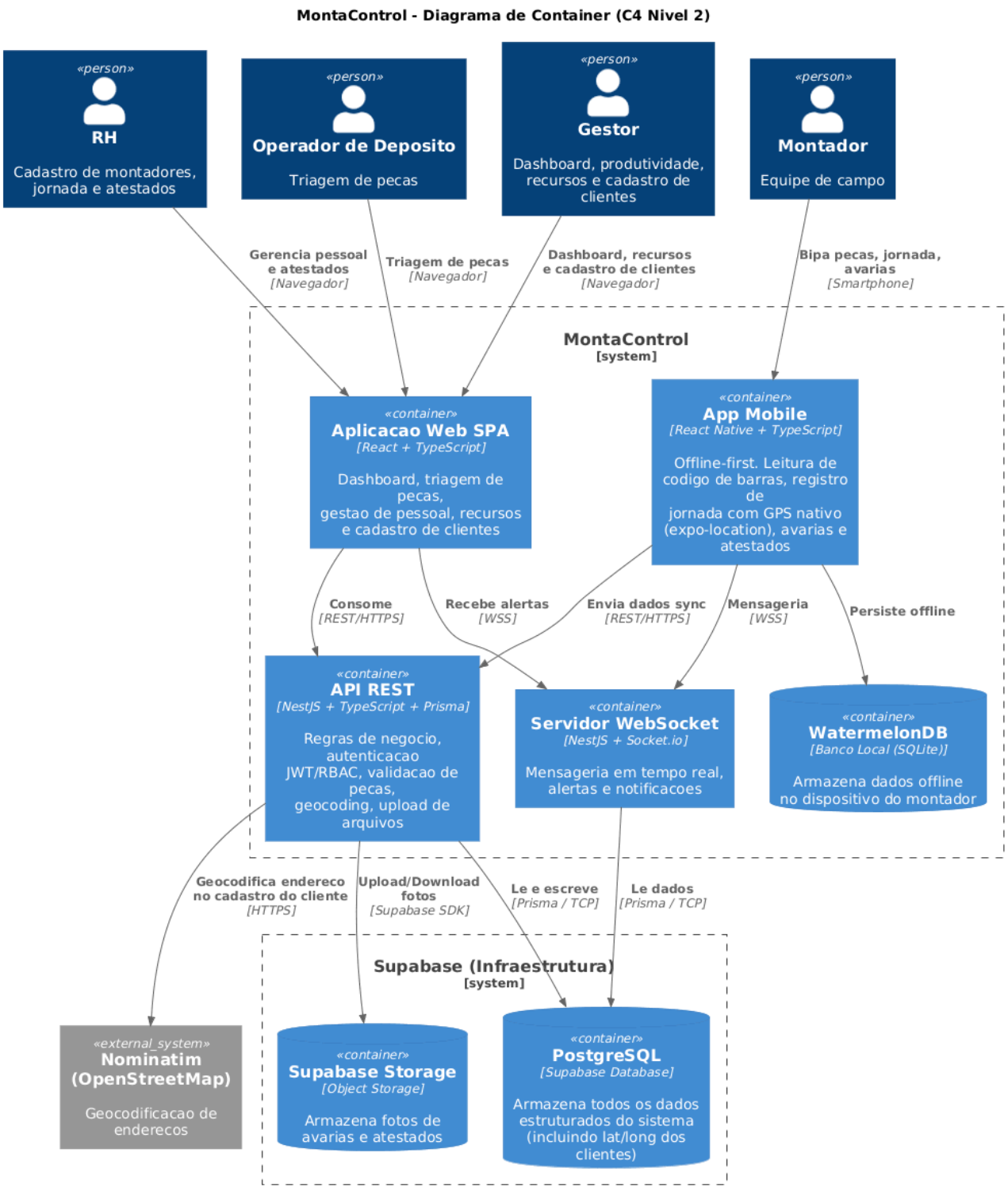



Figura 9. Diagrama de Containers do Sistema MontaControl (Modelo C4: Nível 2).

A Figura 10 detalha os componentes internos da API REST e do Servidor WebSocket, correspondente ao Nível 3 do modelo C4. A API REST organiza os endpoints por contexto de domínio, com um módulo de Autenticação/RBAC que valida tokens JWT e restringe o acesso a dados sensíveis conforme o perfil de cada ator. Os serviços de domínio cobrem as regras de negócio de Peças, Gestão de Pessoal, Gestão de Recursos e Arquivos, todos acessando o banco de dados via Repositório implementado com Prisma ORM. O Servidor WebSocket, composto pelo WebSocket Handler e pelo Serviço de Notificações, gerencia conexões persistentes e despacha alertas em tempo real ao Dashboard gerencial quando eventos críticos são publicados pelos serviços de domínio.

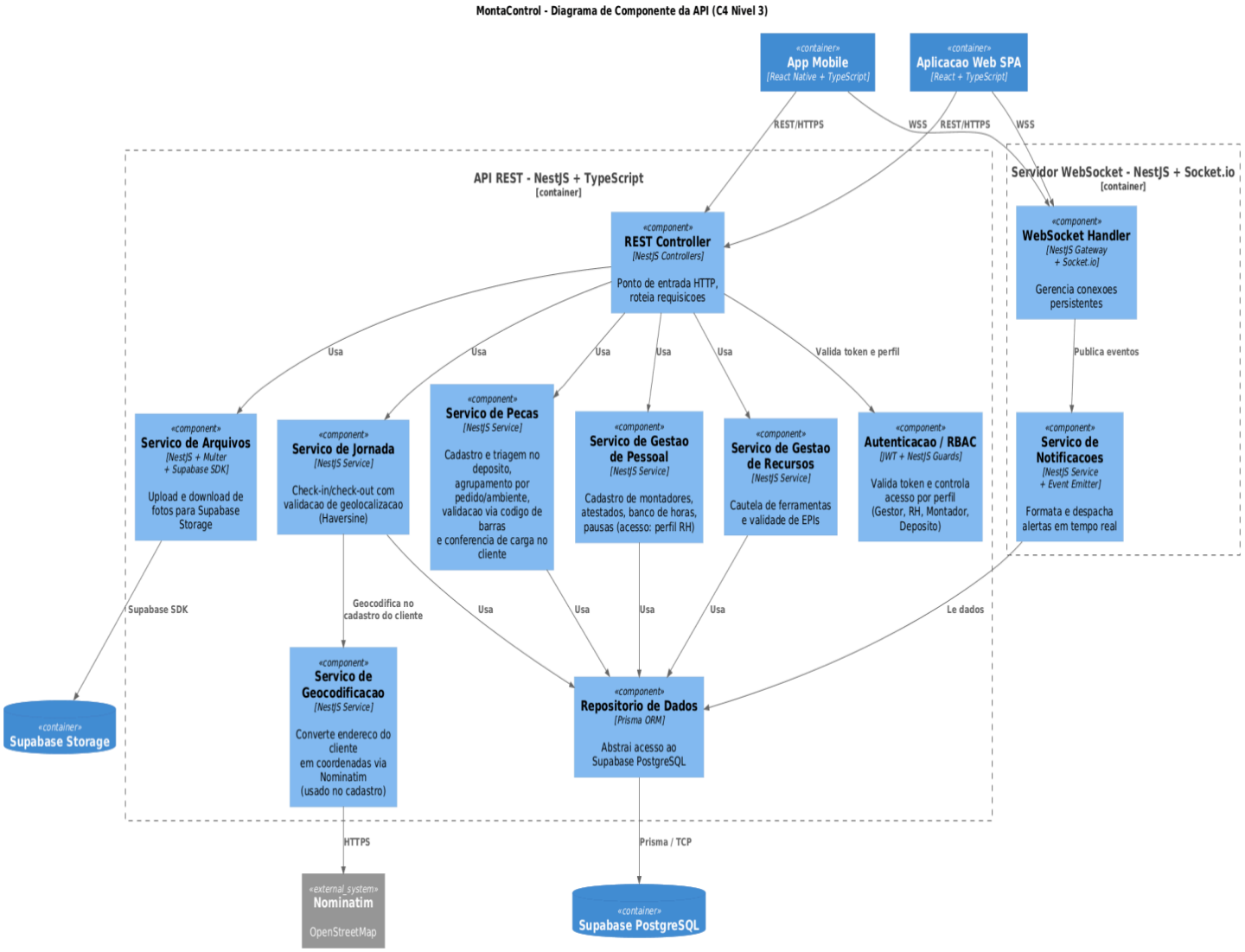



Figura 10. Diagrama de Componentes da API REST e do Servidor WebSocket (Modelo C4: Nível 3).

## 3.4 Diagrama de Componentes e Implantação.

A Figura 11 ilustra o Diagrama de Componentes UML 2.0 do sistema MontaControl, com notação de interfaces requeridas e providas (*lollipop notation*). A arquitetura organiza-se em quatro pacotes. O Client Mobile agrupa os componentes funcionais do montador, coordenados pelo Gerenciador de Sincronização, que persiste dados localmente no WatermelonDB e os sincroniza com a API ao restabelecer conexão. O Client Web reúne os módulos gerenciais, servidos por um Cliente HTTP (Axios) para requisições REST e um Cliente WebSocket (Socket.io) para alertas em tempo real. O subsistema API expõe as interfaces IRestAPI e IWebSocket, conectando-se internamente aos serviços de domínio e ao Repositório via Prisma ORM. A camada Supabase hospeda o PostgreSQL e o Storage, formando a base de persistência do sistema.

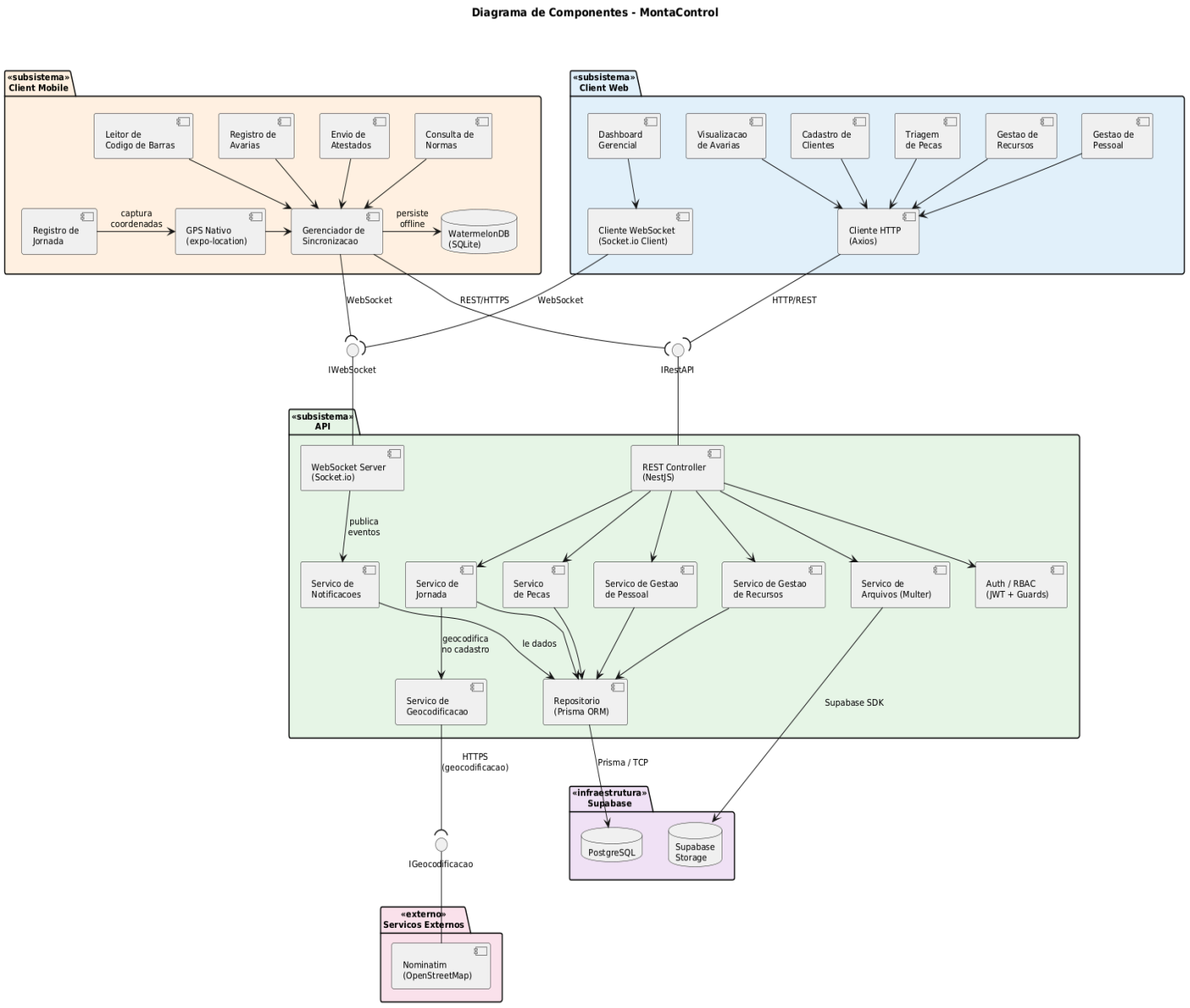



Figura 11. Diagrama de Componentes UML 2.0 do Sistema MontaControl.

A Figura 12 ilustra o Diagrama de Implantação do sistema MontaControl. O dispositivo móvel do montador executa o aplicativo em React Native com WatermelonDB para operação *offline*. As estações de trabalho dos perfis administrativos acessam a Aplicação Web SPA via navegador. O servidor de aplicação concentra a API REST em NestJS e o Servidor WebSocket em Socket.io. Essa camada comunica-se com a infraestrutura no Supabase, que hospeda o PostgreSQL para dados estruturados e o Storage para arquivos. Adicionalmente, a integração com a API do Nominatim (OpenStreetMap) provê a geocodificação utilizada na validação do registro de jornada.

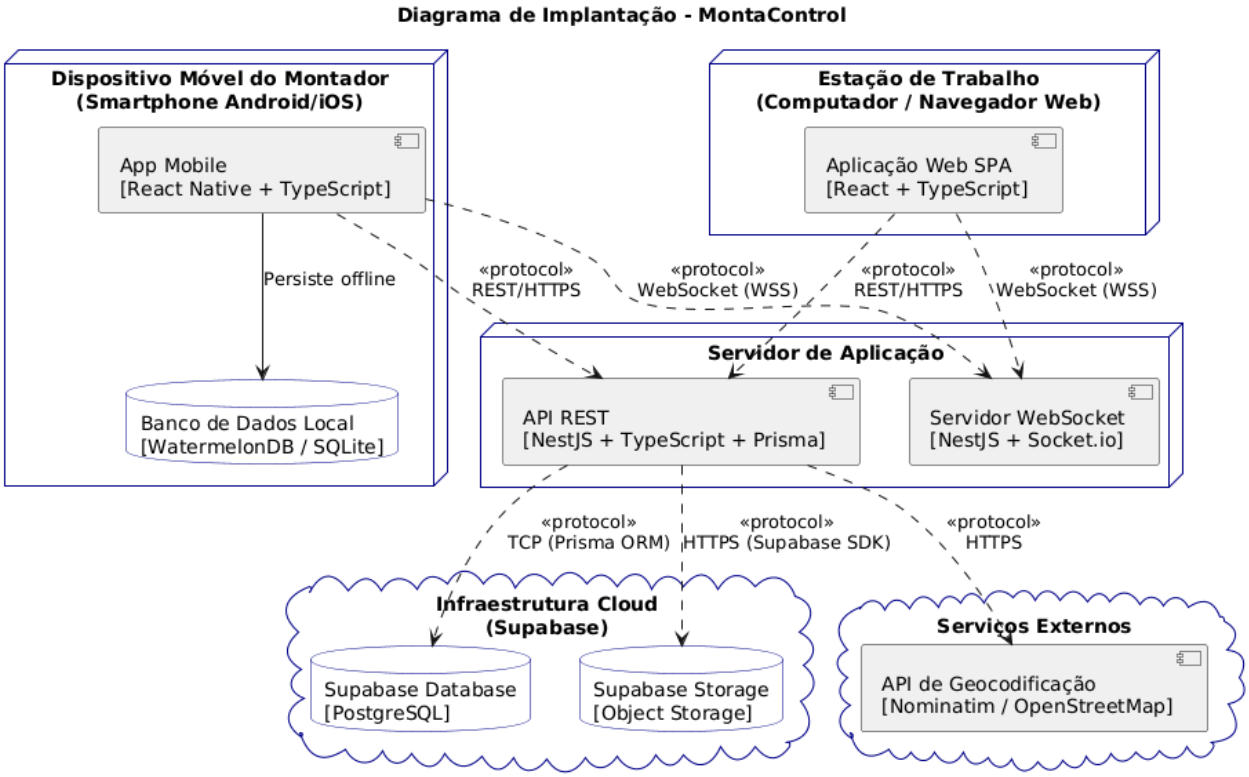



Figura 12. Diagrama de Implantação do Sistema MontaControl.

## 3.5 Diagrama de Estados.

A Figura 13 ilustra o Diagrama de Máquina de Estados referente ao ciclo de vida de uma peça, mapeando as transições desde o recebimento logístico até a conclusão da montagem. O ciclo inicia em Aguardando_Triagem com a chegada da carga da fábrica. Após a leitura e o registro no módulo *web*, a peça transita para Em_Estoque, agrupada por pedido e ambiente. No descarregamento via aplicativo mobile, o fluxo se bifurca: a leitura pode gerar o estado Inconsistente, em caso de erro ou falta, ou levar ao estado Descarregada_Sucesso. A partir deste último, a montagem bem-sucedida atinge o estado final Montagem_Concluida; a identificação de dano leva a peça a Com_Avaria. Tanto a avaria quanto a inconsistência convergem para Aguardando_Reposicao, acionando o Gestor para solicitar reposição à fábrica, o que reinicia o ciclo na triagem.

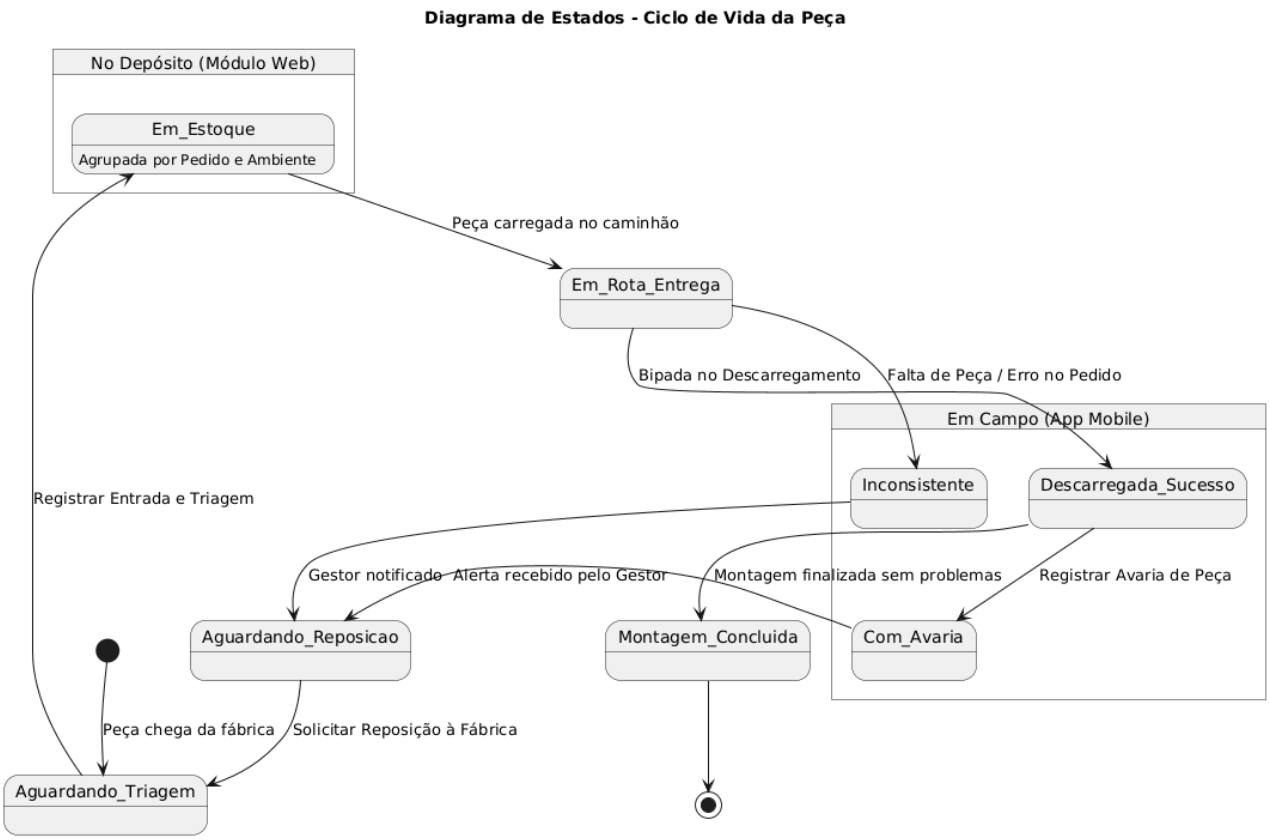



Figura 13. Diagrama de Máquina de Estados: Ciclo de Vida de uma Peça.

A Figura 14 apresenta o Diagrama de Máquina de Estados do ciclo de vida de um atestado médico, modelando a interação assíncrona entre o Ator Montador e o setor de RH. O ciclo inicia quando o montador fotografa o documento pelo aplicativo, criando o registro no estado Criado. Após o *upload* para o Supabase Storage, o artefato transita automaticamente para Pendente_De_Validacao e passa a ser exibido no painel *web*. A decisão do RH determina o encerramento do ciclo: a rejeição atualiza o estado para Rejeitado e dispara uma notificação ao montador; a aprovação evolui o estado para Aprovado e aciona o abono automático no banco de horas.

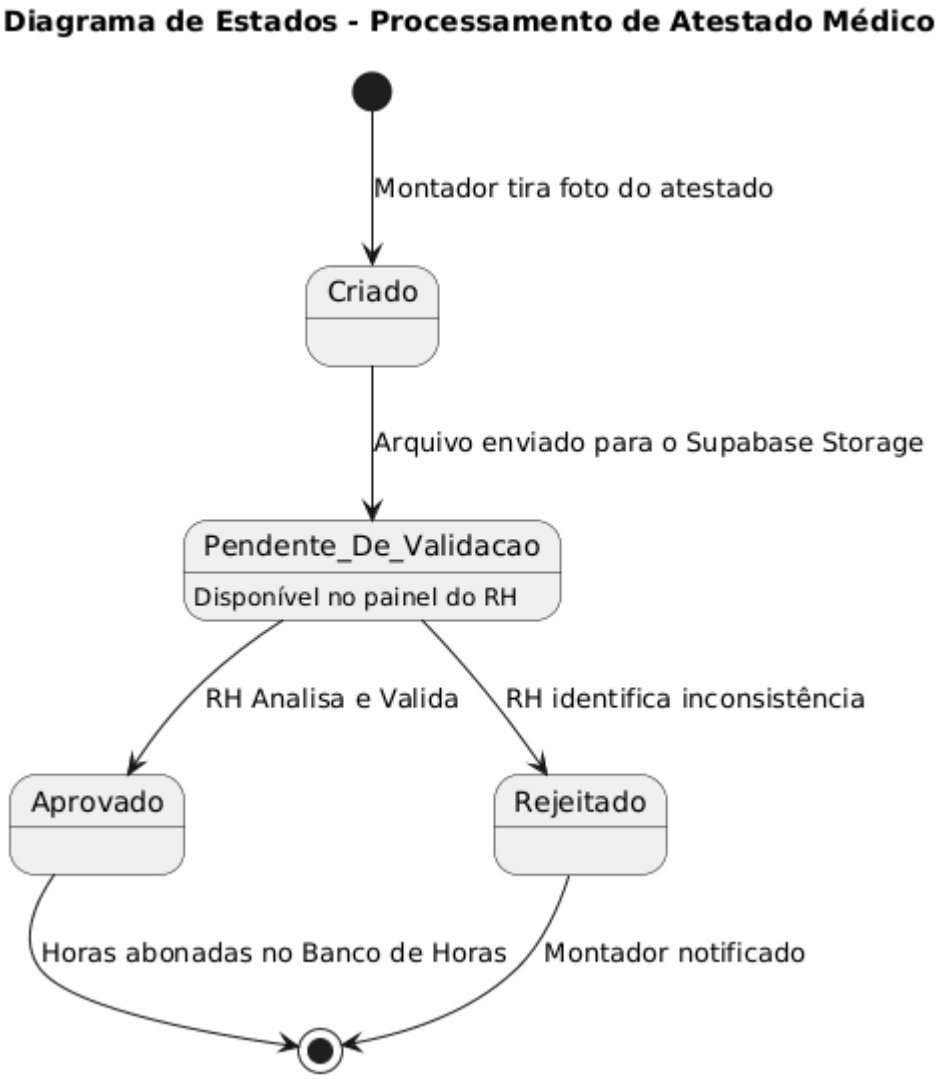



Figura 14. Diagrama de Máquina de Estados: Processamento de Atestado Médico.

# 4. Projeto de Interface com Usuário

Esta seção tem como objetivo principal apresentar e descrever as principais interfaces de usuário desenvolvidas para o sistema MontaControl. O projeto das telas foi estruturado com o propósito de mapear todas as funcionalidades necessárias para o cumprimento dos requisitos do sistema.

## 4.1 Esboço das Interfaces Comuns a Todos os Atores Web

As interfaces apresentadas nesta subseção são comuns a todos os atores que acessam a Aplicação Web. Embora não correspondam diretamente a uma história de usuário específica, são essenciais para conectar os módulos, emitir avisos gerais e proporcionar uma navegação fluida a todos os usuários. As Figuras 15, 16 e 17 ilustram, respectivamente, as telas de Notificação, Perfil e *Login* da aplicação *web*.

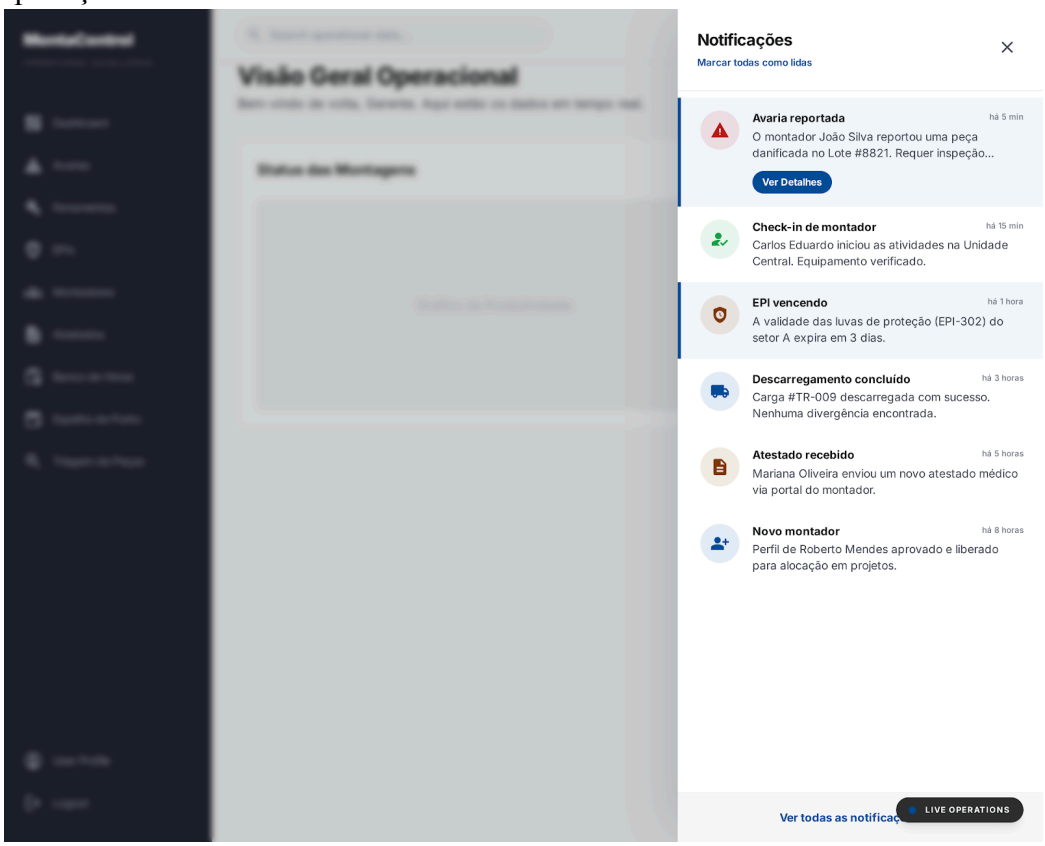



Figura 15. Tela de Notificação (Aplicação *Web*).

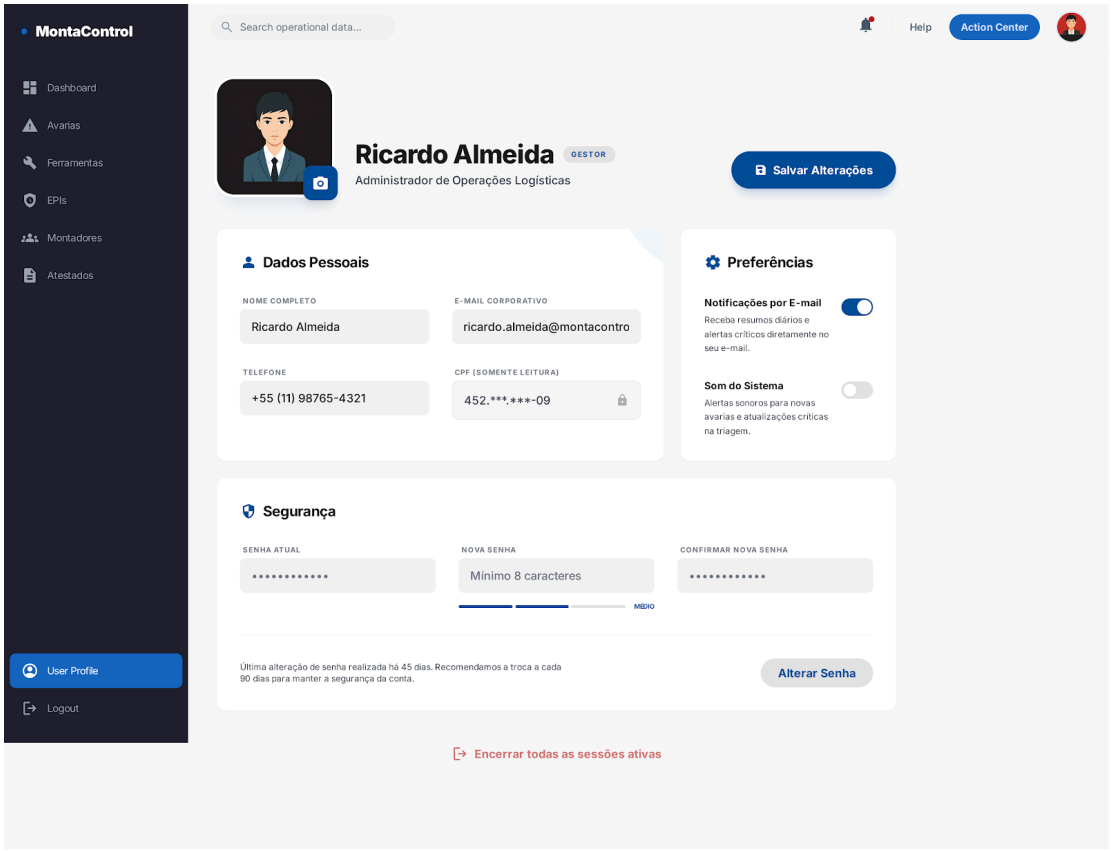



Figura 16. Tela de Perfil (Aplicação *Web*).

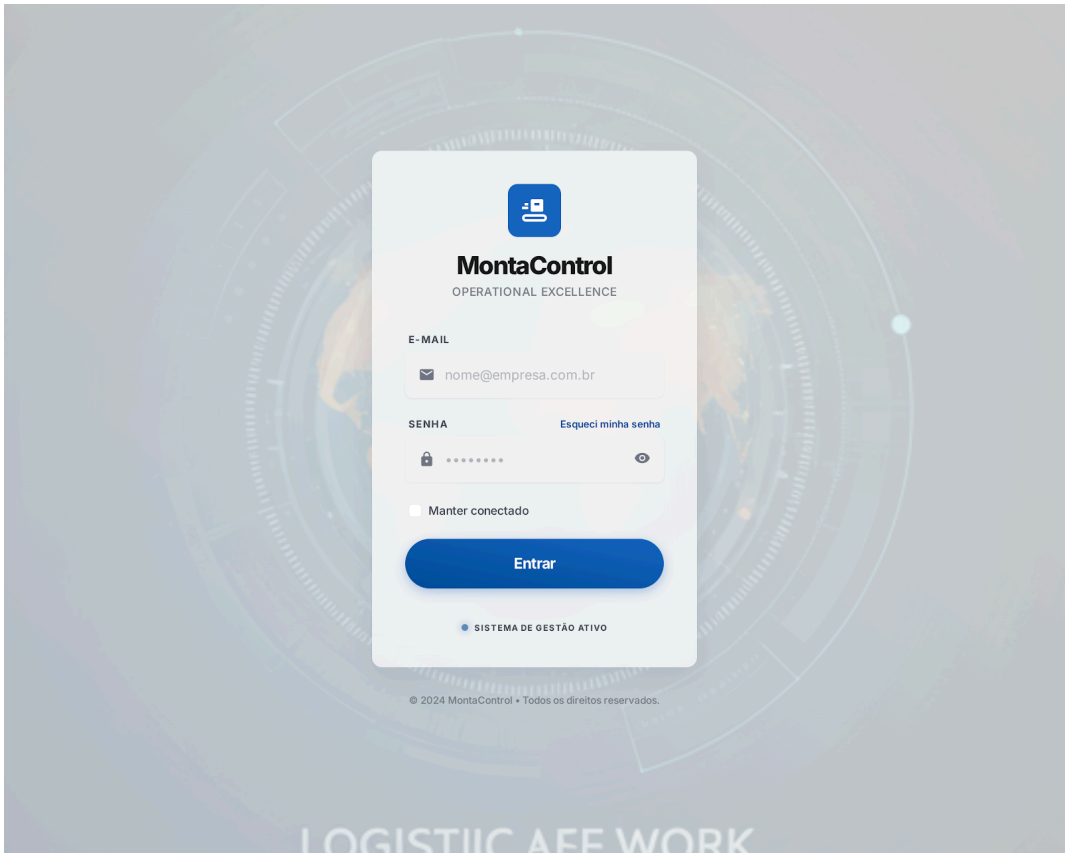
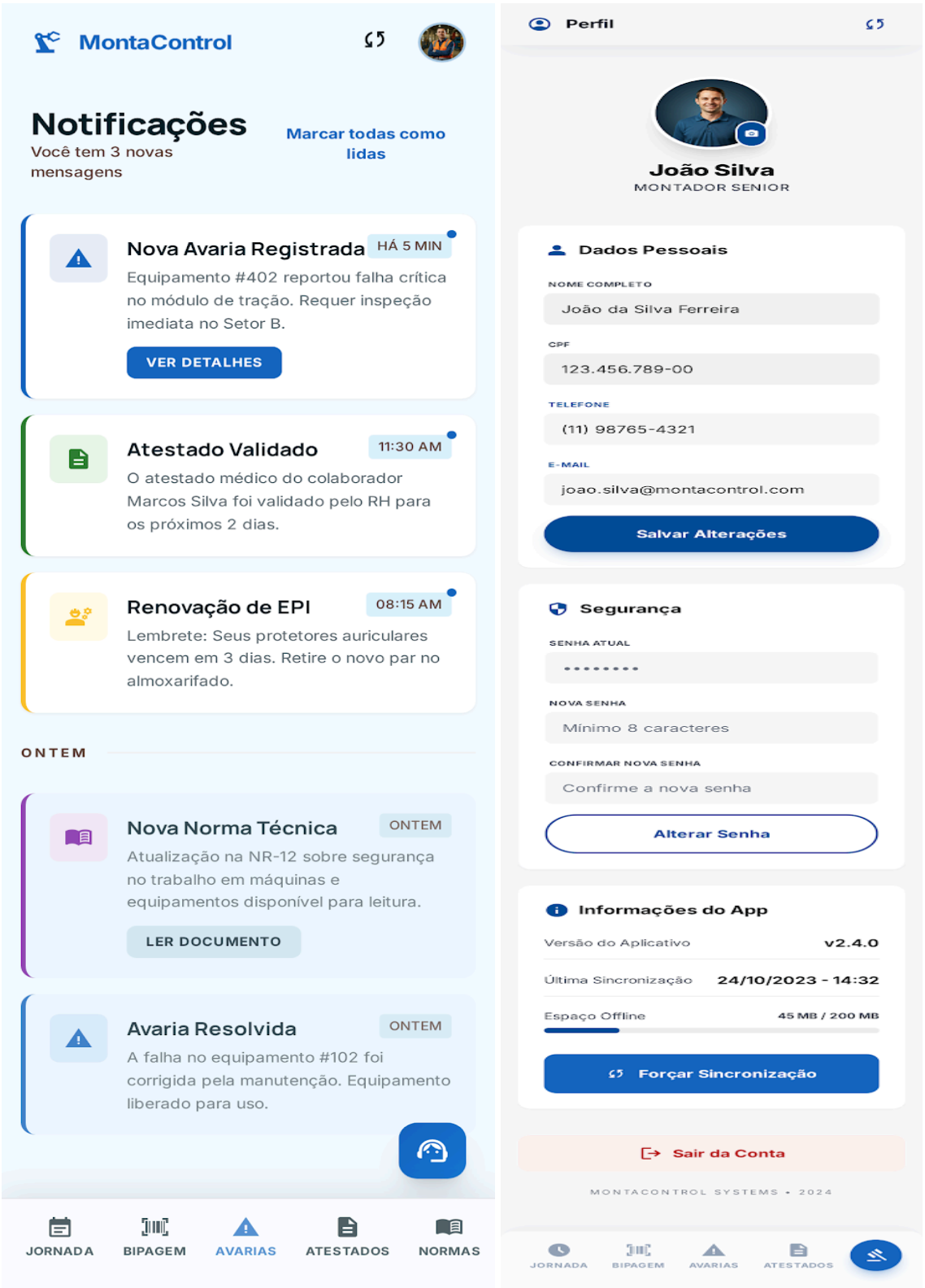


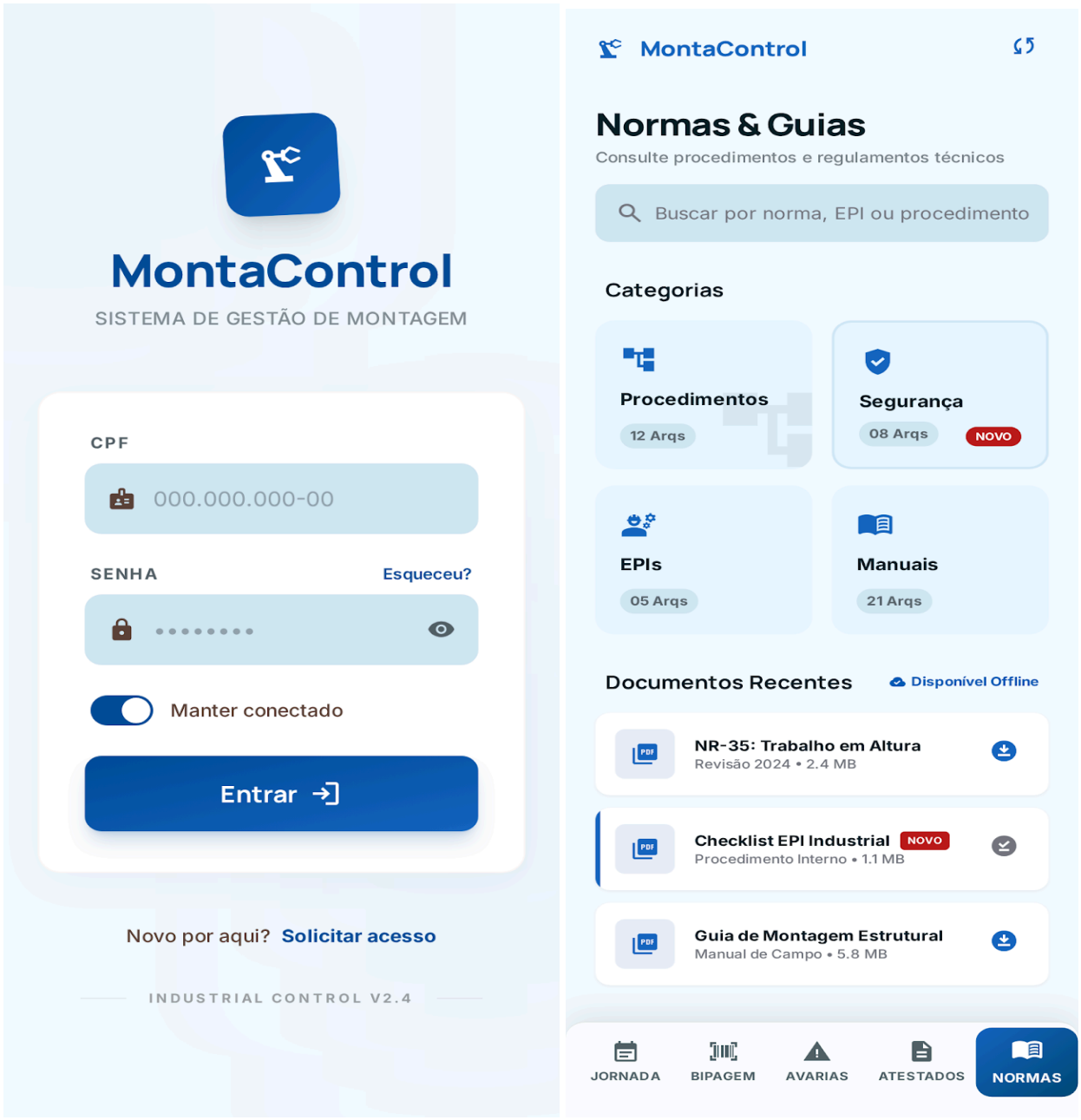

Figura 17. Tela de Login (Aplicação *Web*).

## 4.2 Esboço das Interfaces Usadas pelo Ator Montador

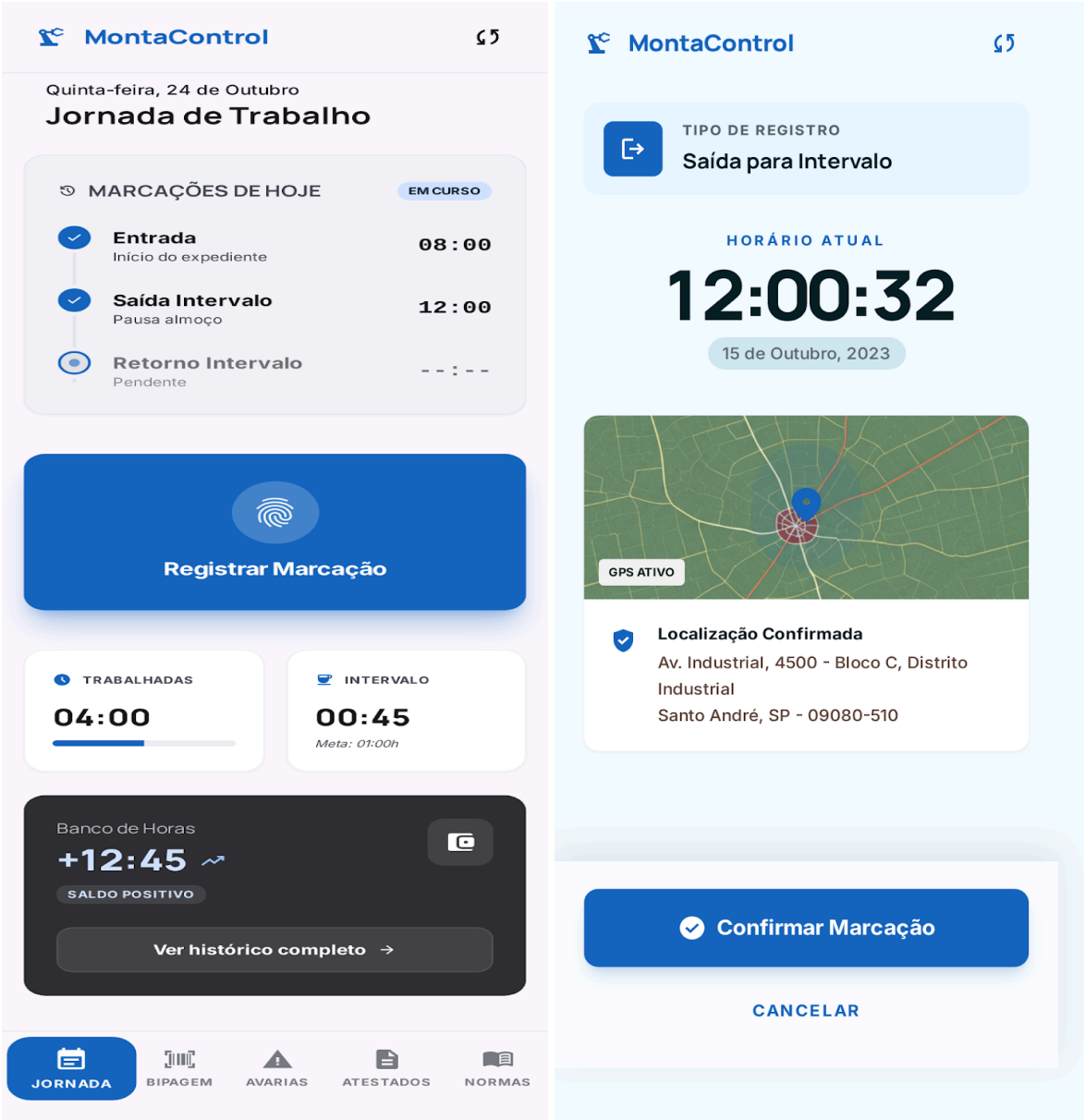
As interfaces descritas nesta subseção contemplam as histórias de usuário US01, US02, US03, US04, US05 e US06. O conjunto de telas oferece ao Ator Montador as ferramentas necessárias para o dia a dia operacional em campo: o registro de ponto com validação de GPS (Figuras 22 e 23), a conferência de peças com suporte *offline* via leitura de código de barras (Figuras 24 e 25), o reporte de avarias com captura fotográfica, o envio de atestados médicos (Figuras 26 e 27), e as telas de consulta de saldo de banco de horas e normas da empresa (Figuras 18, 19, 20 e 21).



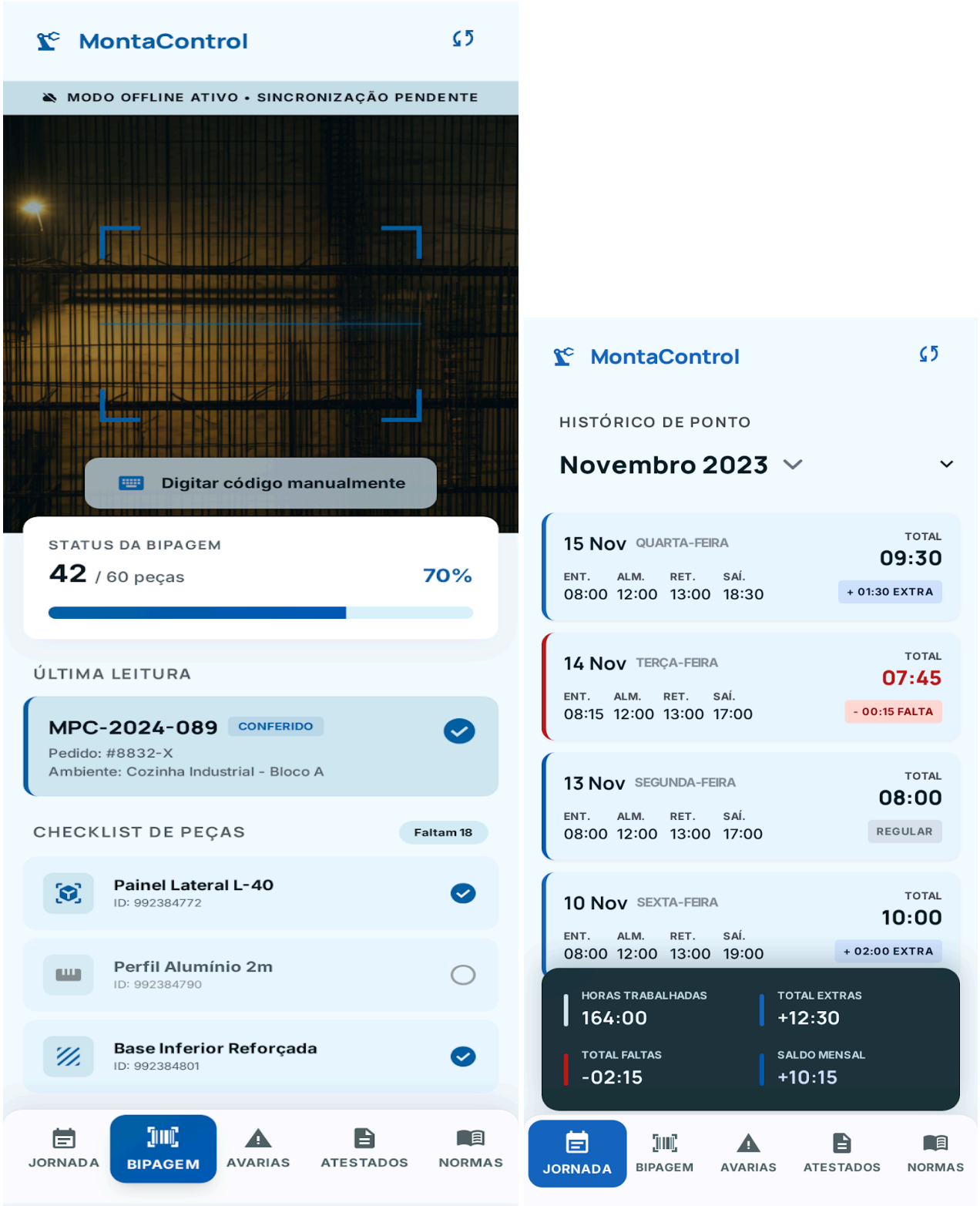

Figuras 18 e 19. Telas de Notificação e Perfil (Aplicativo *Mobile*).




Figuras 20 e 21. Telas de Login e Consulta de Normas (Aplicativo *Mobile*).




Figuras 22 e 23. Telas de Registro de Ponto e Validação de Geolocalização (Aplicativo *Mobile*).




Figuras 24 e 25. Telas de Leitura de Código de Barras e Histórico de Conferência (Aplicativo *Mobile*).







Figuras 26 e 27. Telas de Registro de Avaria e Envio de Atestado Médico (Aplicativo *Mobile*).

## 4.3 Esboço das Interfaces Usadas pelo Ator Operador Logístico

A interface descrita nesta subseção contempla a história de usuário US07. A tela, ilustrada na Figura 28, é direcionada à leitura contínua de códigos de barras pelo Ator Operador Logístico, garantindo que o sistema automatize o cadastro e a triagem das peças de acordo com os pedidos e os ambientes de montagem.

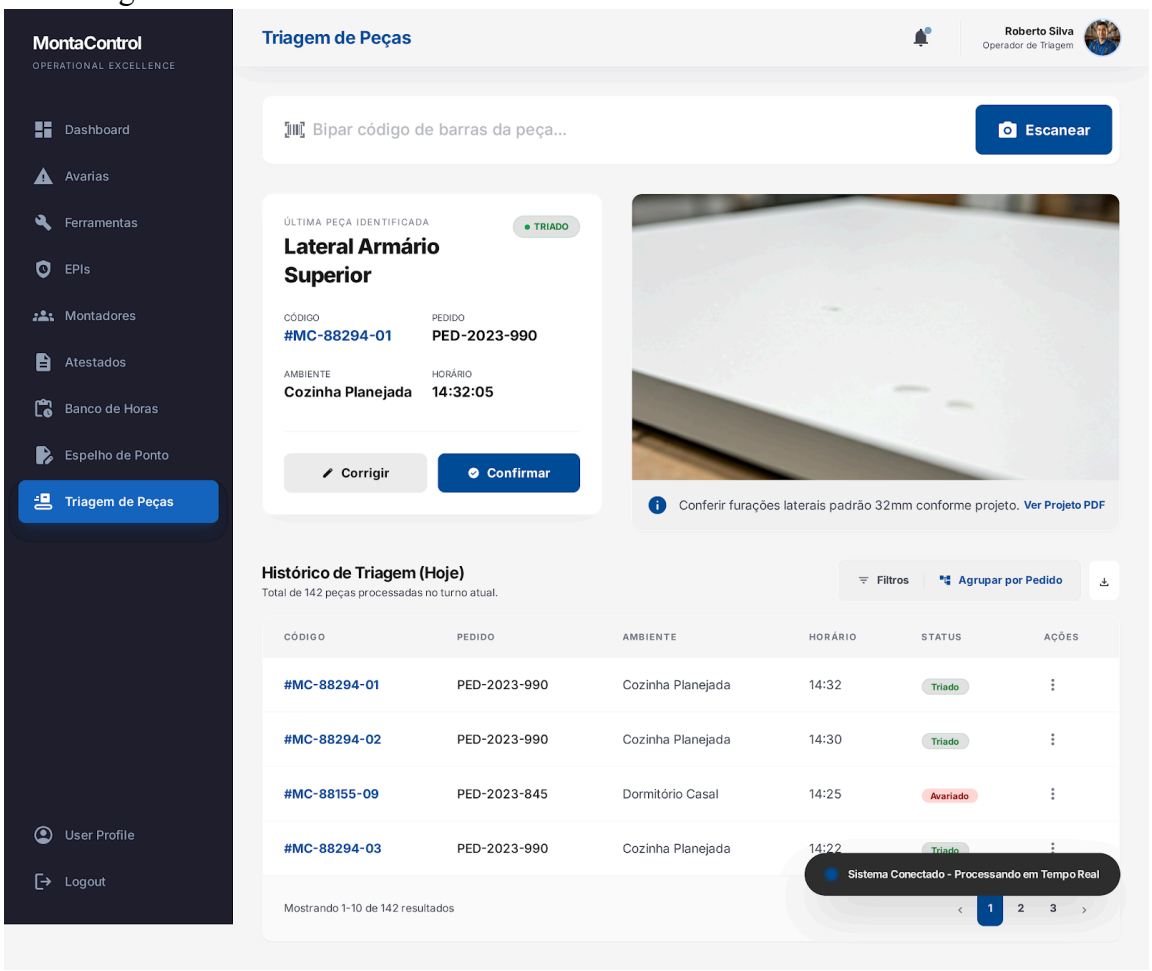



Figura 28. Tela de Triagem de Peças (Aplicação *Web*: Ator Operador Logístico).

## 4.4 Esboço das Interfaces Usadas pelo Ator RH

As interfaces descritas nesta subseção contemplam as histórias de usuário US12, US13 e US14. O conjunto de telas disponibiliza ao Ator Profissional de RH: formulários para o cadastro seguro de novos montadores (Figura 29); módulo de análise e validação de atestados médicos (Figura 30); painel de controle do banco de horas com aprovação de compensações (Figura 31); visualização do espelho de ponto (Figura 32); e tela de gestão de clientes (Figura 33).

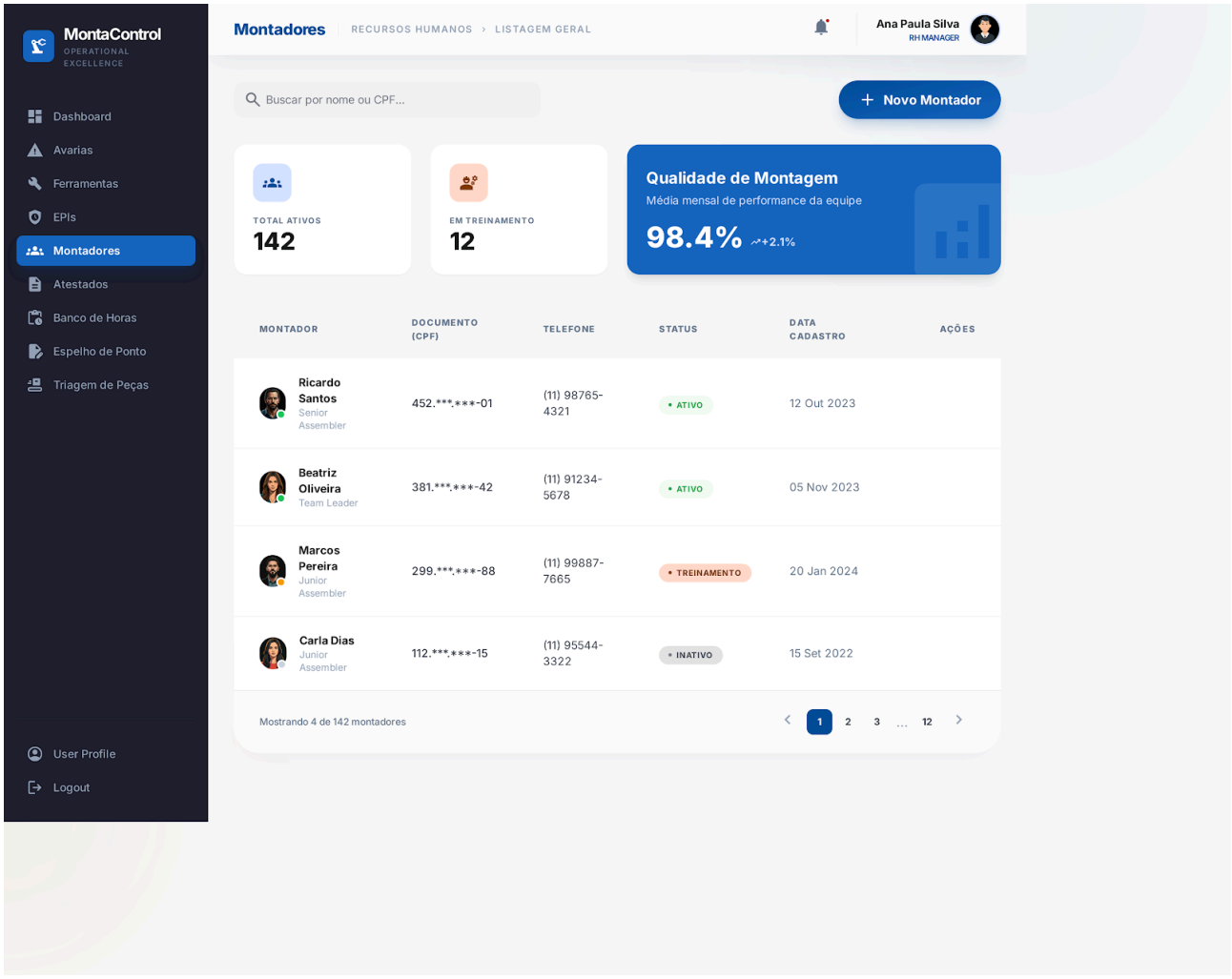



Figura 29. Tela de Listagem e Gestão de Montadores (Aplicação *Web*: Ator RH).

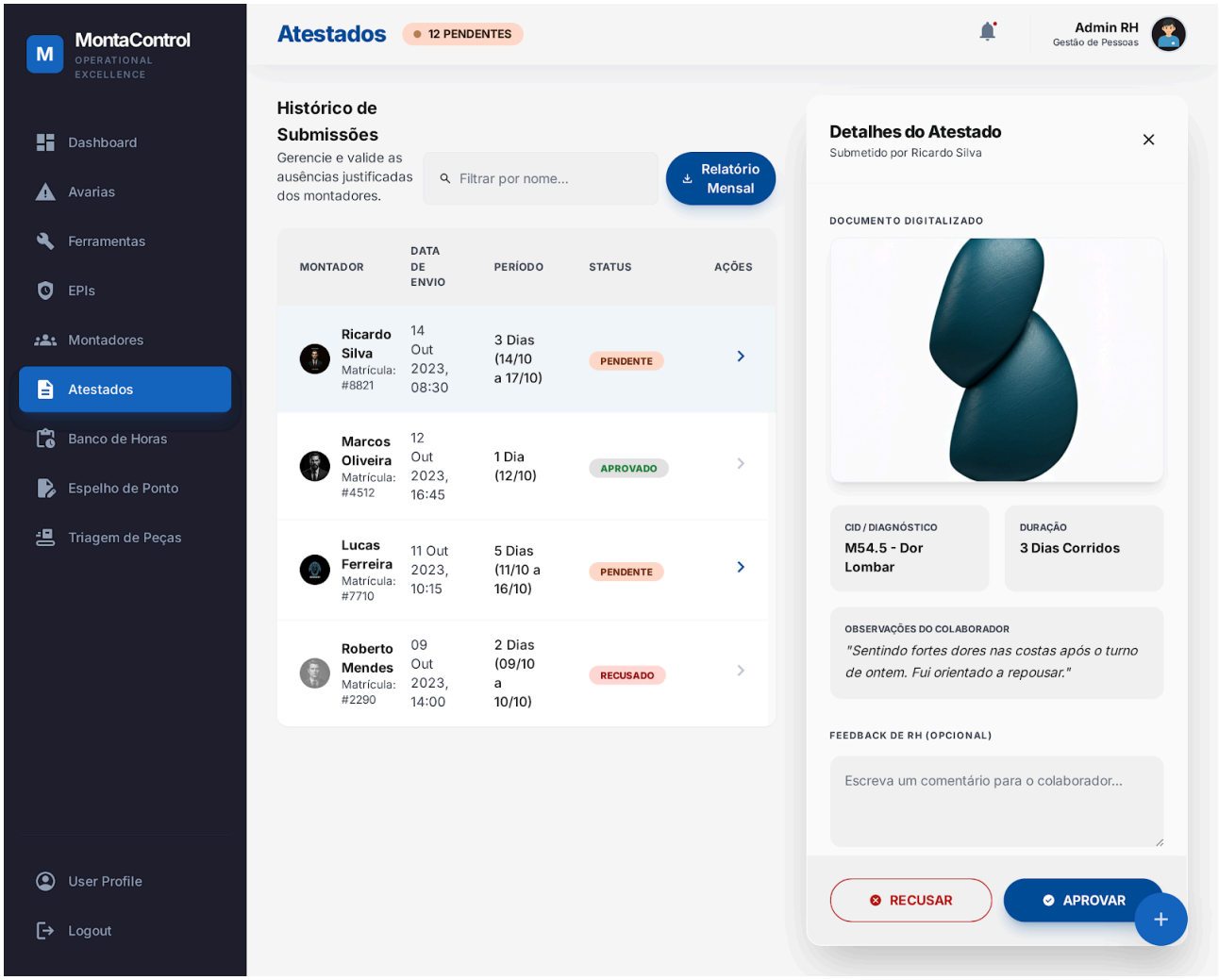



Figura 30. Tela de Gestão e Validação de Atestados Médicos (Aplicação *Web*: Ator RH).

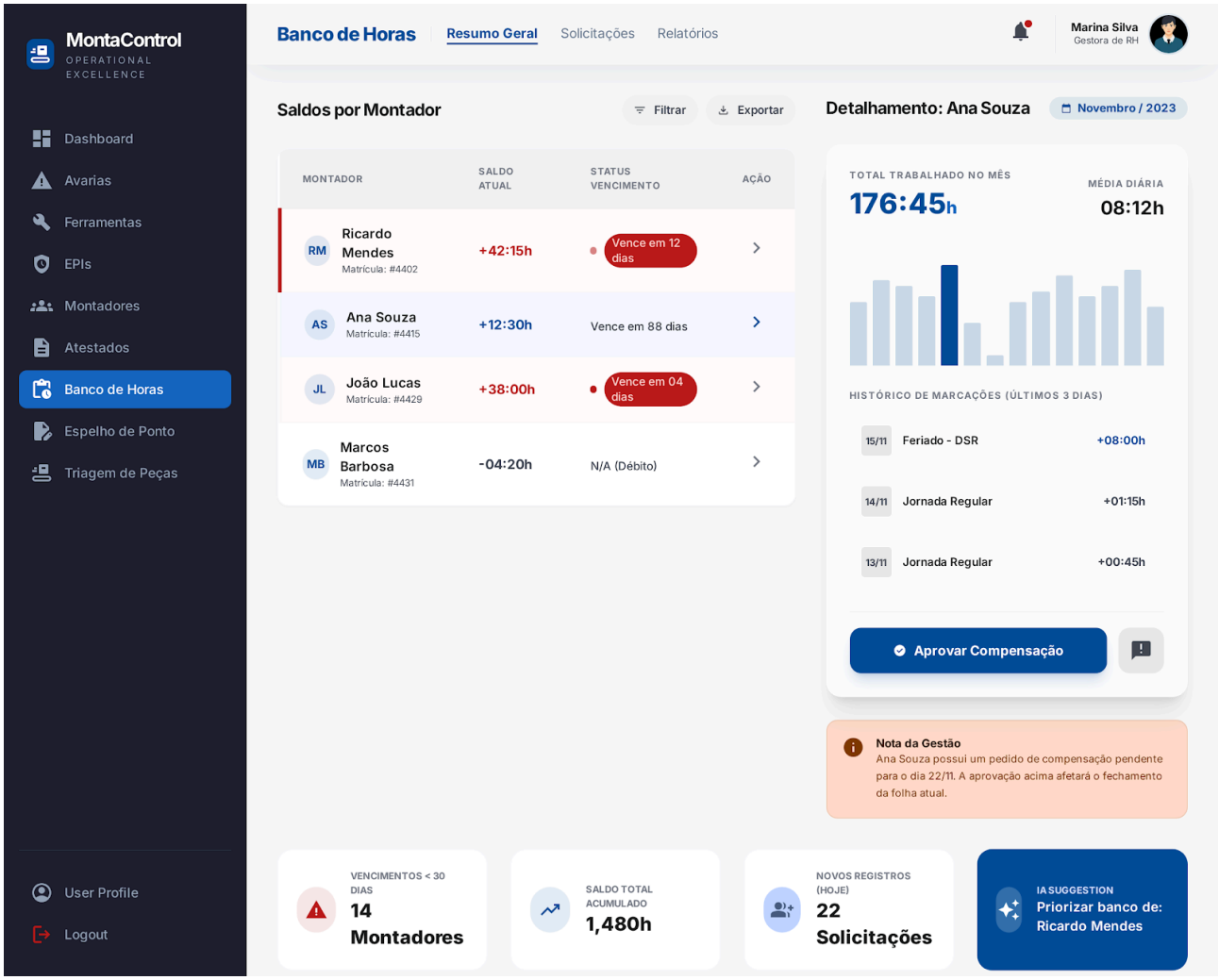



Figura 31. Tela de Controle do Banco de Horas (Aplicação *Web*: Ator RH).

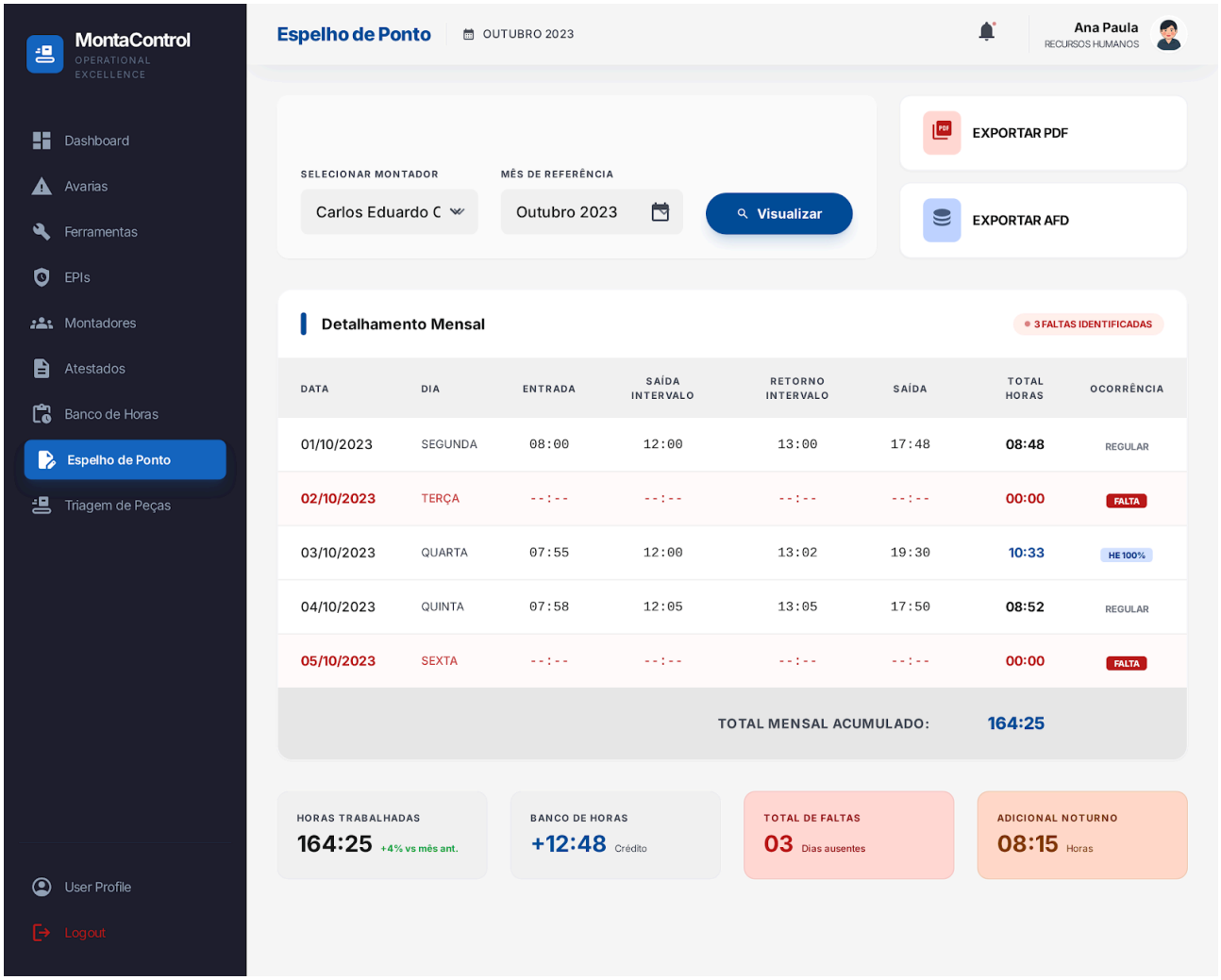



Figura 32. Tela de Visualização do Espelho de Ponto (Aplicação *Web*: Ator RH).

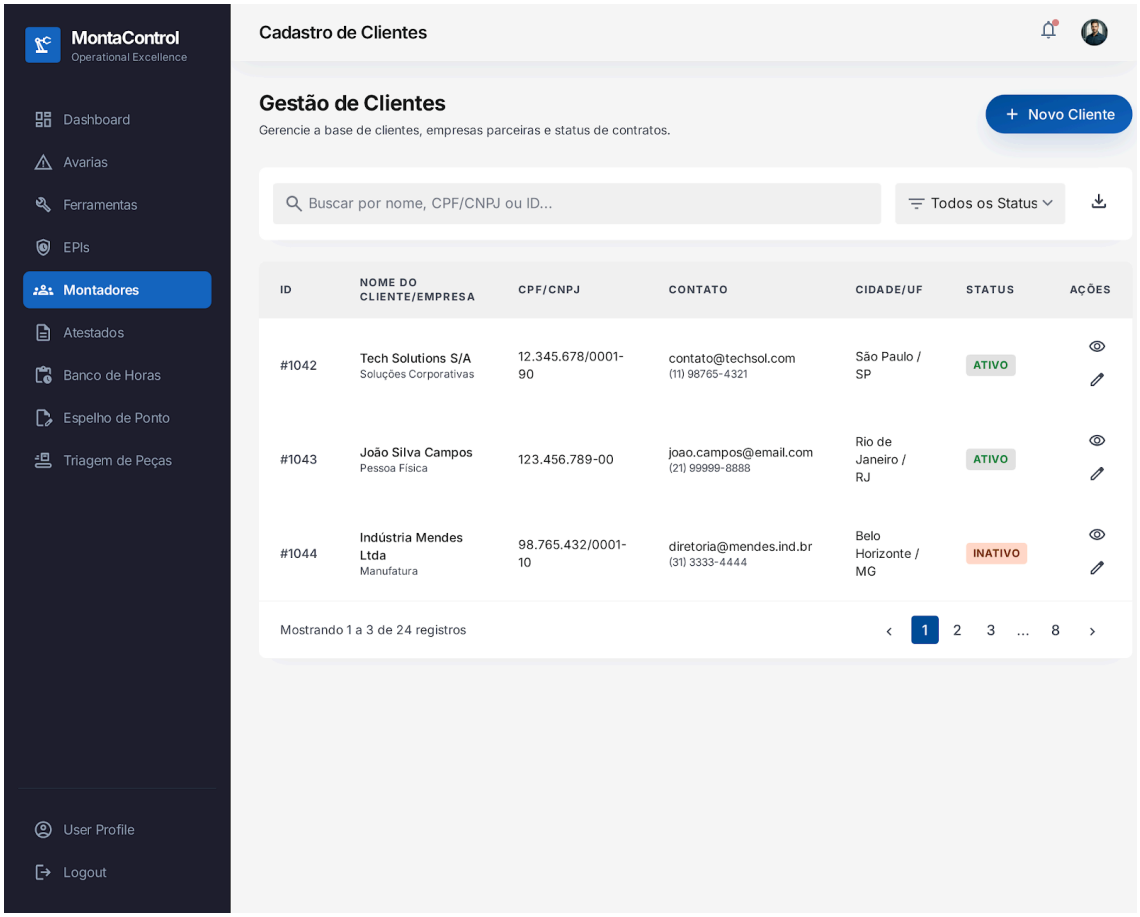



Figura 33. Tela de Gestão de Clientes (Aplicação *Web*: Ator Gestor da Franquia).

## 4.5 Esboço das Interfaces Usadas pelo Ator Gestor

As interfaces descritas nesta subseção contemplam as histórias de usuário US08, US09, US10 e US11. O conjunto de telas disponibiliza ao Ator Gestor da Franquia: o dashboard gerencial em tempo real com indicadores de produtividade (Figura 37); o módulo de detalhamento e tratamento de avarias reportadas em campo (Figuras 34 e 36); e o módulo de controle patrimonial de ferramentas via cautela e monitoramento do vencimento dos EPIs da equipe (Figura 35).

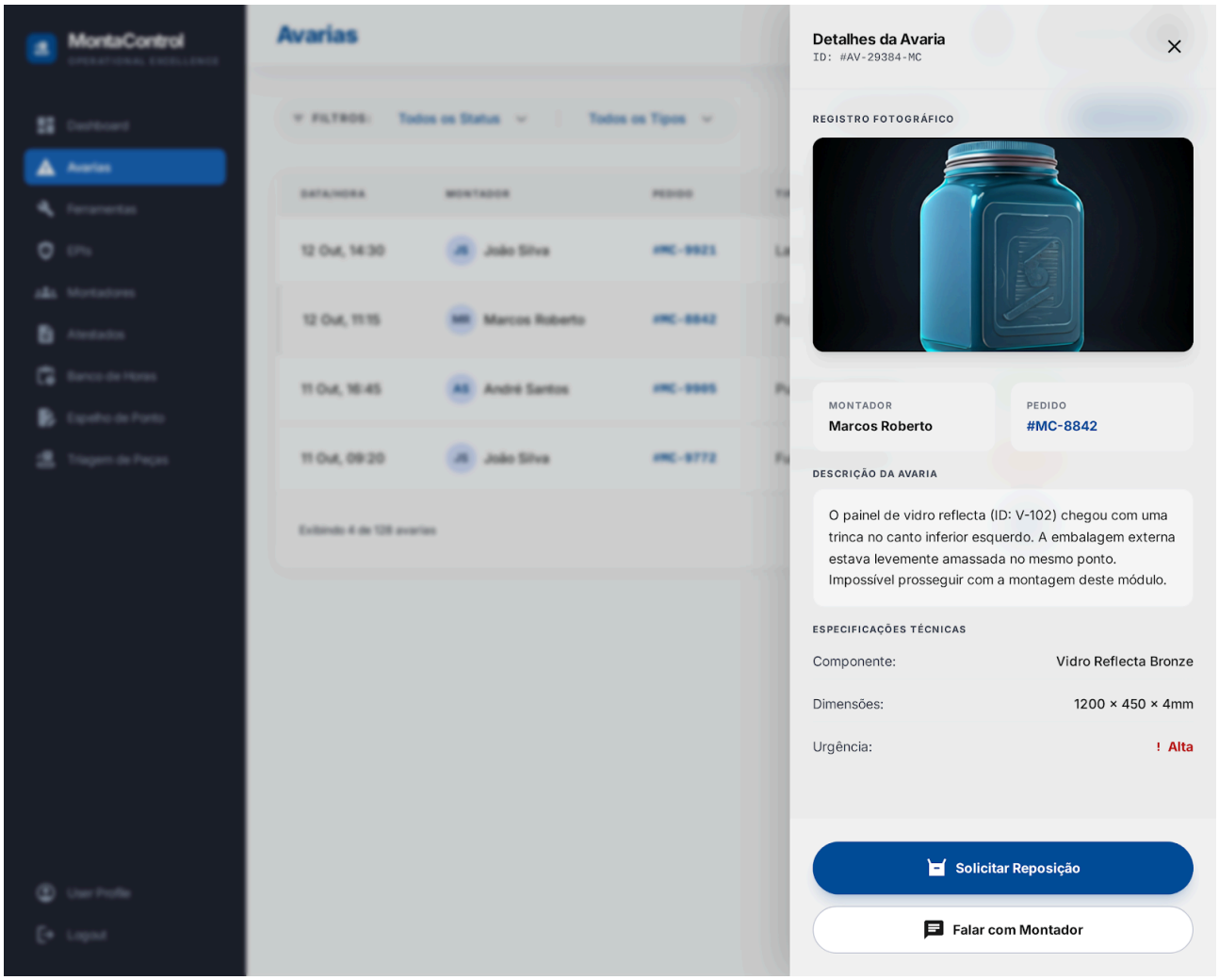



Figura 34. Tela de Detalhes de Avaria (Aplicação *Web*: Ator Gestor da Franquia).

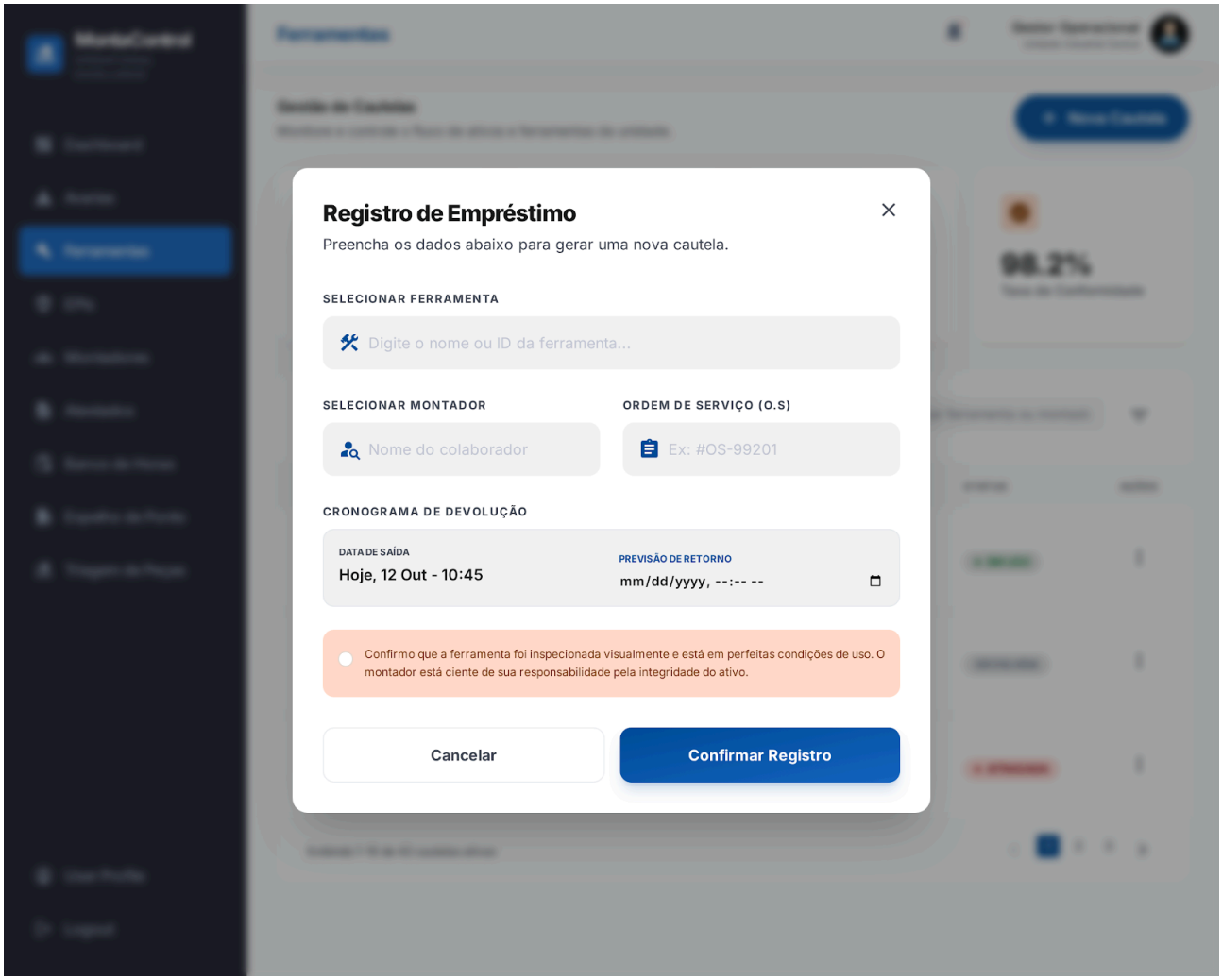



Figura 35. Tela de Registro de Empréstimo de Ferramenta (Aplicação *Web*: Ator Gestor da Franquia).

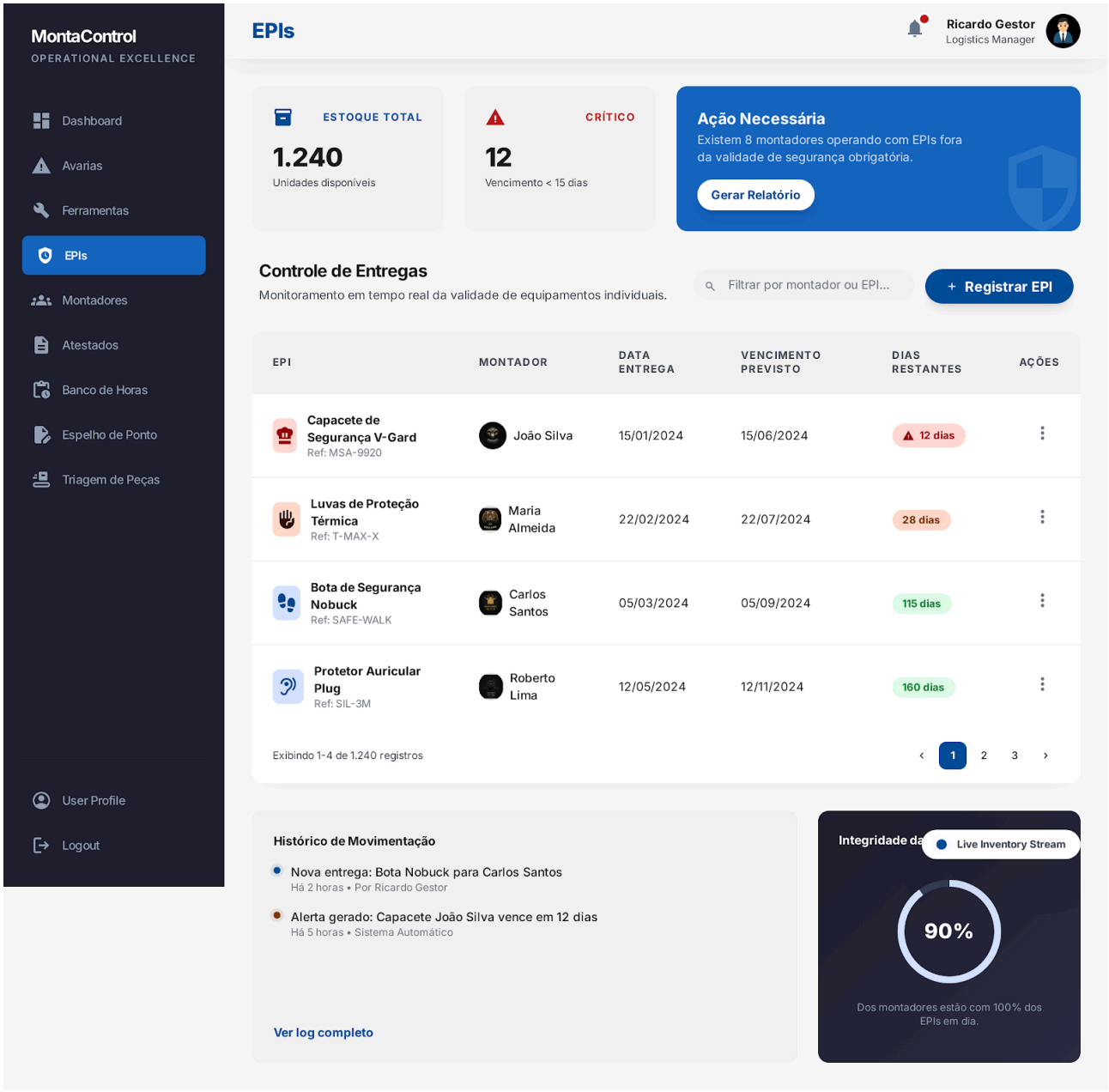



Figura 36. Tela de Listagem de Avarias (Aplicação *Web*: Ator Gestor da Franquia).

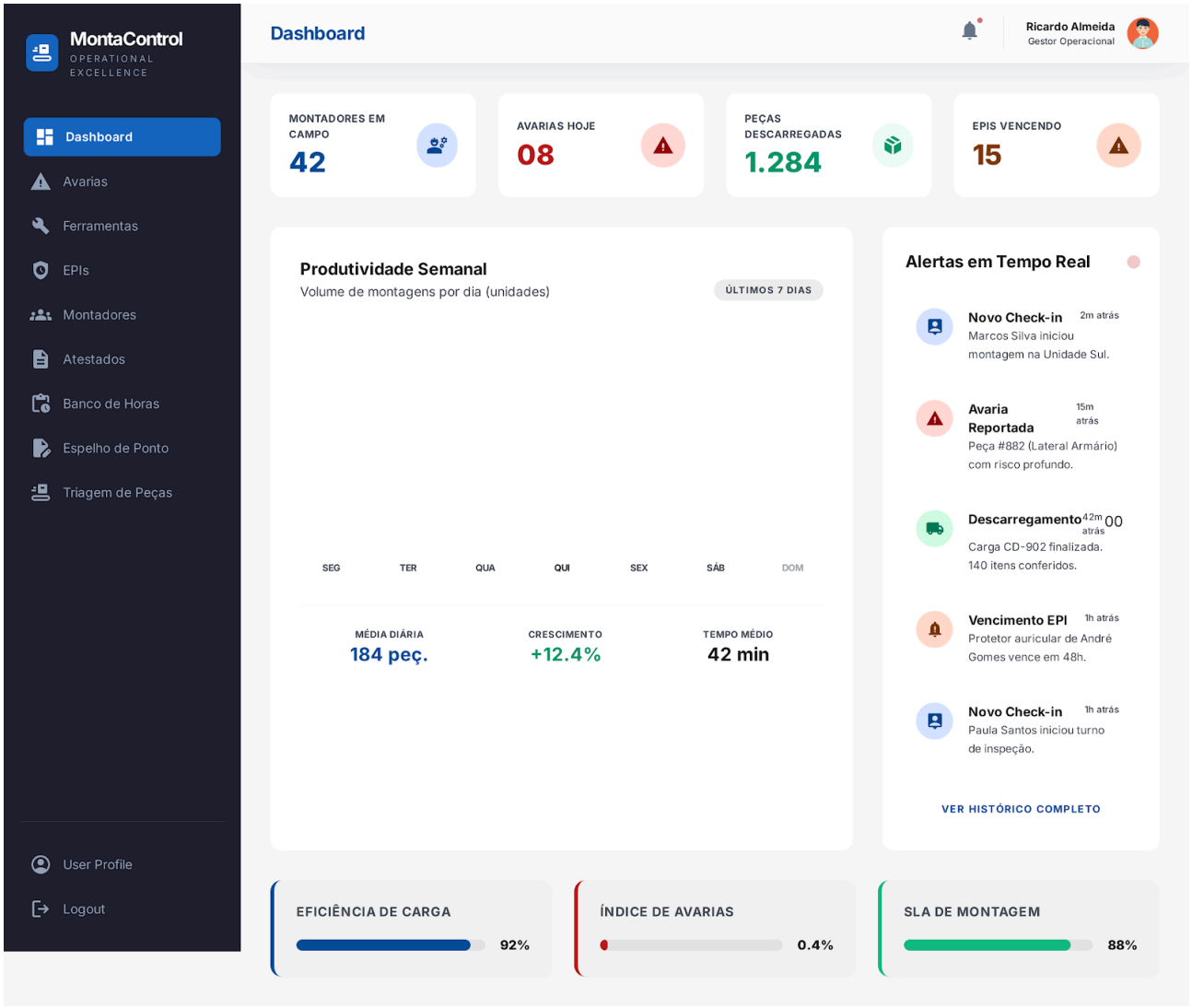



Figura 37. Tela do Dashboard Gerencial em Tempo Real (Aplicação *Web*: Ator Gestor da Franquia).

# 5. Glossário e Modelos de Dados

Esta seção apresenta o glossário estruturado dos atributos do banco de dados do MontaControl, com foco em segurança e preservação de histórico. A modelagem padroniza o uso de chaves primárias numéricas, chaves únicas para identificadores de negócio externos e o tipo enum para estados fixos do sistema. A integridade dos dados é garantida pela prioridade dada à auditoria temporal, à rastreabilidade dos responsáveis por ações sensíveis e à adoção de exclusão lógica (*soft delete*) para evitar a perda de informações.

| **Padrões Globais do Sistema** | | |
|---|---|---|
| **Atributo** | **Formato** | **Descrição** |
| id | Int | Identificador único e sequencial de cada registro (Chave Primária / PK). |
| [entidade]_id | Int | Chave Estrangeira (FK) utilizada para relacionar tabelas (ex: cliente_id, pedido_id, usuario_id). |
| [acao]_por | Int | Chave Estrangeira (FK) focada em rastreabilidade de quem executou uma ação (ex: validado_por, triada_por, conferida_por, registrada_por). |
| ativo | Boolean | Indicador utilizado para exclusão lógica (soft delete), preservando o histórico do registro. |
| created_at | Timestamp | Data e hora exata em que o registro foi criado no banco de dados. |
| updated_at | Timestamp | Data e hora exata da última modificação do registro. |
| **Gestão de Clientes e Pedidos** | | |
| **Atributo** | **Formato** | **Descrição** |
| nome | String | Nome completo do cliente cadastrado. |
| endereco, cidade, estado, cep | String | Conjunto de dados que compõem o endereço físico do cliente. |
| latitude, longitude | Decimal | Coordenadas geográficas exatas do endereço do cliente. |
| telefone | String | Número de contato do cliente. |
| numero_pedido | String | Identificador único de negócio do pedido (Chave Única / UK). |
| status (Pedido) | Enum | Situação atual do pedido logístico (AGUARDANDO, EM_TRIAGEM, RECEBIDO, EM_ENTREGA, ENTREGUE). |
| titulo | String | Nome ou título de uma Norma cadastrada no sistema. |

| | | |
|---|---|---|
| conteudo | Text | Texto detalhado com as diretrizes ou regras da Norma. |
| **Usuários e Jornada de Trabalho** | | |
| **Atributo** | **Formato** | **Descrição** |
| nome | String | Nome do colaborador. |
| email | String | Endereço de e-mail usado para autenticação (Chave Única / UK). |
| senha_hash | String | Criptografia (hash) da senha de acesso do usuário. |
| perfil | Enum | Define o nível de permissão e acesso no sistema (GESTOR, RH, MONTADOR, DEPOSITO). |
| checkin_latitude, checkin_longitude | Decimal | Coordenadas de geolocalização capturadas no início da jornada de trabalho. |
| checkout_latitude, checkout_longitude | Decimal | Coordenadas de geolocalização capturadas no fim da jornada de trabalho. |
| status (Jornada) | Enum | Estado da jornada diária do montador (EM_ANDAMENTO, PAUSADA, FINALIZADA). |
| inicio, fim (Pausa) | Timestamp | Registros exatos de tempo para os intervalos dentro de uma jornada. |
| mes, ano | Int | Referência temporal para o fechamento do Banco de Horas. |
| horas_trabalhadas | Decimal | Total de horas efetivamente trabalhadas no mês. |
| horas_previstas | Decimal | Total de horas esperadas/contratadas para o mês. |
| saldo | Decimal | Diferença entre horas trabalhadas e previstas (banco de horas). |
| **Saúde, Segurança e Recursos Humanos** | | |
| **Atributo** | **Formato** | **Descrição** |
| data_inicio, data_fim | Date | Período de validade do atestado médico apresentado. |
| status (Atestado) | Enum | Situação da avaliação do atestado pelo RH (PENDENTE, VALIDADO, REJEITADO). |
| motivo_rejeicao | Text | Justificativa preenchida pelo RH caso o atestado seja invalidado. |
| validado_at | Timestamp | Data e hora em que a validação do atestado foi concluída. |
| nome, descricao (Tipo EPI) | String / Text | Identificação e detalhamento da categoria do Equipamento de Proteção Individual. |
| validade_meses | Int | Tempo de vida útil padrão previsto para o tipo de EPI. |
| data_entrega | Date | Data em que o EPI foi entregue ao colaborador. |
| data_validade | Date | Data limite de uso do EPI, baseada na data de entrega e validade em meses. |
| **Controle de Montagem (Ambientes, Peças e Avarias)** | | |

| Atributo | Formato | Descrição |
|---|---|---|
| nome (Ambiente) | String | Nome do espaço físico onde a montagem ocorre (ex: Cozinha, Quarto). |
| status (Ambiente) | Enum | Progresso da montagem daquele ambiente específico (PENDENTE, EM_PROGRESSO, COMPLETO). |
| codigo_barras | String | Identificador físico da peça bipada no sistema (Chave Única / UK). |
| descricao | String / Text | Detalhamento textual de uma peça, avaria ocorrida ou tipo de avaria. |
| status (Peça) | Enum | Acompanhamento do fluxo da peça (TRIADA, CARREGADA, ENTREGUE). |
| status (Avaria) | Enum | Acompanhamento da resolução do problema (REPORTADA, EM_ANALISE, REPOSICAO_SOLICITADA, RESOLVIDA). |
| storage_path | String | Caminho de destino do arquivo da foto (atestado ou avaria) no storage (ex: Supabase). |
| mime_type | String | Formato do arquivo da imagem anexada (ex: image/jpeg, image/png). |
| tamanho_bytes | Int | Tamanho do arquivo armazenado em bytes. |
| **Gestão de Ferramentas e Cautelas** | | |
| **Atributo** | **Formato** | **Descrição** |
| nome, descricao | String / Text | Nomenclatura e especificações técnicas de um tipo de ferramenta. |
| codigo_patrimonio | String | Número de tombamento físico da ferramenta pertencente à empresa (Chave Única / UK). |
| disponivel | Boolean | Sinaliza se a ferramenta está no depósito pronta para ser emprestada. |
| data_retirada | Timestamp | Data e hora em que o colaborador pegou a ferramenta (Cautela). |
| data_devolucao_prevista | Date | Prazo estipulado para o retorno da ferramenta. |
| data_devolucao_real | Timestamp | Data e hora exata em que a ferramenta foi devolvida ao depósito. |
| devolvida | Boolean | Checkbox lógico de conclusão do empréstimo da cautela. |

Tabela 4. Glossário de Atributos e Definições do Banco de Dados do Sistema MontaControl.

A Figura 38 ilustra as vinte entidades da aplicação distribuídas em sete domínios de negócio, evidenciando as práticas de modelagem relacional adotadas no projeto. O esquema destaca-se pela adoção do padrão Tipo/Item para separar metadados de registros físicos e pelo isolamento de arquivos em tabelas satélites normalizadas, preparadas para a integração com o Supabase Storage. Todo o esquema do banco de dados é mapeado e gerenciado pelo Prisma ORM (Mapeamento Objeto-Relacional, do inglês *Object-Relational Mapping*), o que assegura o desenvolvimento com tipagem estática segura e o controle confiável das migrações.

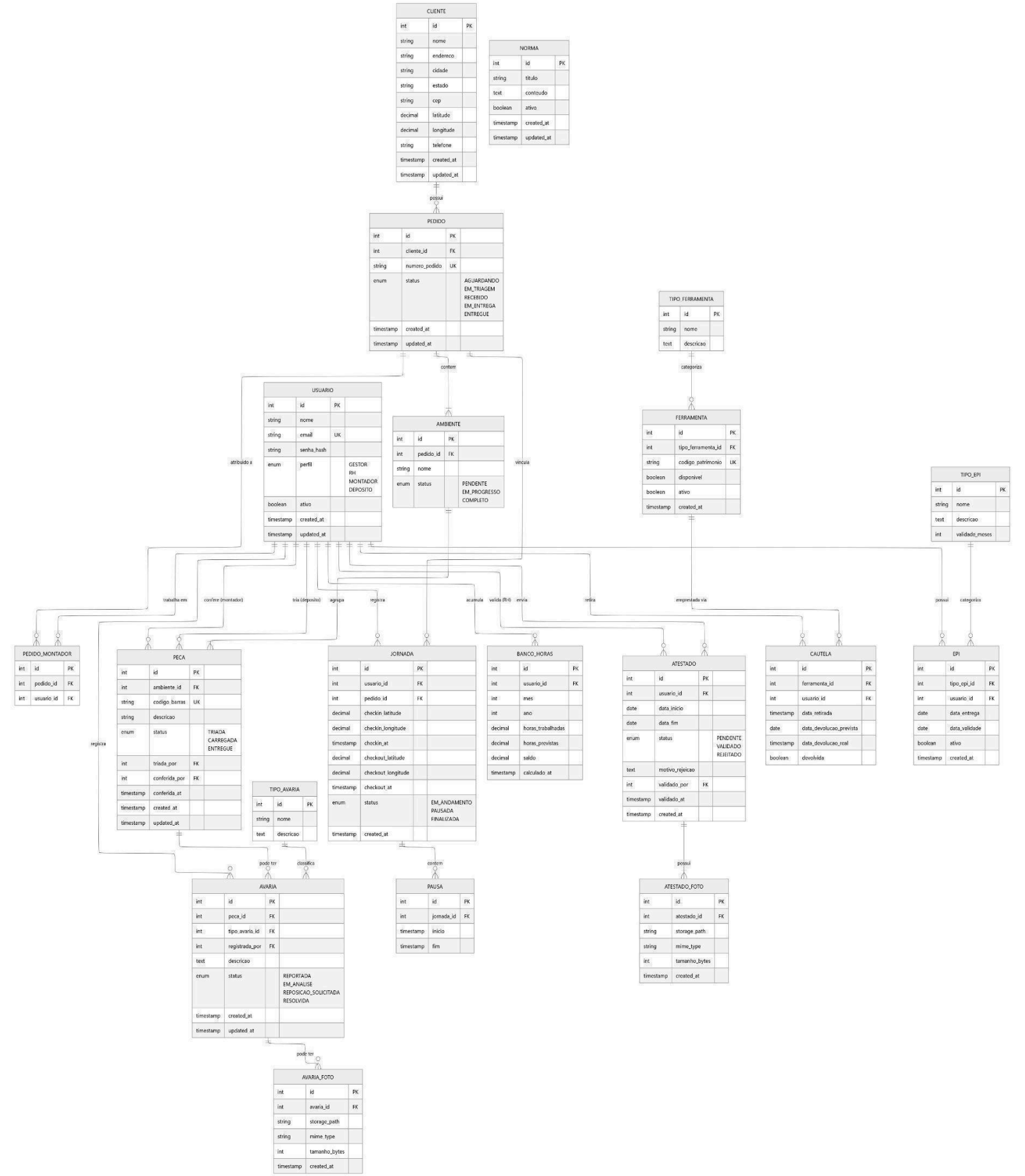

CLIENTE
int id PK
string nome
string endereco
string cidade
string estado
string cep
decimal latitude
decimal longitude
string telefone
timestamp created_at
timestamp updated_at
NORMA
int id PK
string titulo
text conteudo
boolean ativo
timestamp created_at
timestamp updated_at
possui
PEDIDO
int id PK
int cliente_id FK
string numero_pedido UK
enum status AGUARDANDO EM_TRIAGEM RECEBIDO EM_ENTREGA ENTREGUE
timestamp created_at
timestamp updated_at
TIPO_FERRAMENTA
int id PK
string nome
text descricao
contem
categoriza
USUARIO
int id PK
string nome
string email UK
string senha_hash
enum perfil GESTOR RH MONTADOR DEPOSITO
boolean ativo
timestamp created_at
timestamp updated_at
AMBIENTE
int id PK
int pedido_id FK
string nome
enum status PENDENTE EM_PROGRESSO COMPLETO
FERRAMENTA
int id PK
int tipo_ferramenta_id FK
string codigo_patrimonio UK
boolean disponivel
boolean ativo
timestamp created_at
TIPO_EPI
int id PK
string nome
text descricao
int validade_meses
atribuido a
vincula
trabalha em
confere (montador)
tria (deposito)
agrupa
registra
acumula
valida (RH)
envia
retira
emprestada via
possui
categoriza
PEDIDO_MONTADOR
int id PK
int pedido_id FK
int usuario_id FK
PECA
int id PK
int ambiente_id FK
string codigo_barras UK
string descricao
enum status TRIADA CARREGADA ENTREGUE
int triada_por FK
int conferida_por FK
timestamp conferida_at
timestamp created_at
timestamp updated_at
JORNADA
int id PK
int usuario_id FK
int pedido_id FK
decimal checkin_latitude
decimal checkin_longitude
timestamp checkin_at
decimal checkout_latitude
decimal checkout_longitude
timestamp checkout_at
enum status EM_ANDAMENTO PAUSADA FINALIZADA
timestamp created_at
BANCO_HORAS
int id PK
int usuario_id FK
int mes
int ano
decimal horas_trabalhadas
decimal horas_previstas
decimal saldo
timestamp calculado_at
ATESTADO
int id PK
int usuario_id FK
date data_inicio
date data_fim
enum status PENDENTE VALIDADO REJEITADO
text motivo_rejeicao
int validado_por FK
timestamp validado_at
timestamp created_at
CAUTELA
int id PK
int ferramenta_id FK
int usuario_id FK
timestamp data_retirada
date data_devolucao_prevista
timestamp data_devolucao_real
boolean devolvida
EPI
int id PK
int tipo_epi_id FK
int usuario_id FK
date data_entrega
date data_validade
boolean ativo
timestamp created_at
registra
TIPO_AVARIA
int id PK
string nome
text descricao
pode ter
classifica
contem
possui
AVARIA
int id PK
int peca_id FK
int tipo_avaria_id FK
int registrada_por FK
text descricao
enum status REPORTADA EM_ANALISE REPOSICAO_SOLICITADA RESOLVIDA
timestamp created_at
timestamp updated_at
PAUSA
int id PK
int jornada_id FK
timestamp inicio
timestamp fim
ATESTADO_FOTO
int id PK
int atestado_id FK
string storage_path
string mime_type
int tamanho_bytes
timestamp created_at
pode ter
AVARIA_FOTO
int id PK
int avaria_id FK
string storage_path
string mime_type
int tamanho_bytes
timestamp created_at

Figura 38. Diagrama de Entidade-Relacionamento (DER) do Banco de Dados do Sistema MontaControl.

# 6. Casos de Teste

A validação do sistema MontaControl é conduzida por meio de dois conjuntos de casos de teste, apresentados nas Seções 6.1 e 6.2. Os testes de aceitação (TA) confrontam o sistema com as necessidades levantadas no Documento de Visão, aferindo se cada requisito funcional foi implementado conforme o esperado. Os testes de integração (TI) examinam os pontos de comunicação entre os subsistemas, verificando se o fluxo de dados percorre corretamente os diferentes componentes da arquitetura. Cada caso de teste especifica: o estado inicial que o sistema deve apresentar antes da execução (pré-condições), os passos realizados durante a execução (ações) e a resposta esperada ao seu término (resultado esperado).

## 6.1 Testes de Aceitação

Os testes de aceitação têm como ponto de partida as necessidades descritas no Documento de Visão, que por sua vez derivam dos requisitos funcionais do sistema. Para cada necessidade, são formulados três cenários de teste distintos, cobrindo o caminho esperado, um desvio de comportamento e uma situação de contorno relevante para o domínio. As quatro necessidades originais do Documento de Visão foram preservadas e complementadas por necessidades adicionais identificadas ao longo do projeto, totalizando oito necessidades (N) mapeadas a seguir:

**N1.** Acesso seguro e autenticado ao sistema.

**N2.** Acompanhamento gerencial e de produtividade em tempo real.

**N3.** Controle operacional centralizado de RH, EPIs e ferramentas.

**N4.** Garantia de assertividade no descarregamento de peças no cliente.

**N5.** Comunicação instantânea e rastreável de avarias.

**N6.** Registro e validação da jornada de trabalho com geolocalização.

**N7.** Triagem e rastreabilidade de peças no depósito.

**N8.** Envio e proteção de atestados médicos.

Cada tabela de aceitação registra sete informações: o identificador, formado pelo código da necessidade seguido do número sequencial do cenário (por exemplo, N2T1); a necessidade que o teste busca cobrir; o caso de teste, que descreve o cenário em linguagem objetiva; as pré-condições, que definem o estado que o sistema deve apresentar antes de iniciar a execução; os dados de entrada fornecidos durante o teste; a sequência de ações realizadas; e o resultado esperado, que determina o critério de aprovação ou reprovação do cenário. Ao todo, são 24 testes de aceitação distribuídos nas Tabelas 5 a 28, com três cenários por necessidade.

A necessidade N1 trata do acesso seguro e autenticado ao sistema, sendo considerada transversal pois condiciona o uso de todas as demais funcionalidades. O cenário N1T1 (Tabela 5) avalia o fluxo

de autenticação bem-sucedido com credenciais válidas. N1T2 (Tabela 6) verifica se credenciais inválidas são corretamente rejeitadas. N1T3 (Tabela 7) testa o mecanismo de controle de acesso baseado em perfis (RBAC), assegurando que cada ator acesse apenas os recursos autorizados ao seu perfil.

| | |
|---|---|
| **Identificador** | N1T1 |
| **Necessidade** | N1: Acesso seguro e autenticado ao sistema. |
| **Caso de Teste** | Verificar autenticação com credenciais válidas (base para acesso de todos os atores ao sistema). |
| **Pré-condições** | Usuário cadastrado e ativo no sistema. |
| **Dados de Entrada** | E-mail e senha válidos. |
| **Ações** | 1. Acessar a tela de login.<br>2. Inserir e-mail e senha válidos.<br>3. Confirmar o acesso. |
| **Resultado Esperado** | Usuário autenticado e redirecionado conforme o perfil (Gestor, RH, Montador ou Operador Logístico). |

Tabela 5. Teste de aceitação N1T1: Verificar autenticação com credenciais válidas (base para todos os atores).

| | |
|---|---|
| **Identificador** | N1T2 |
| **Necessidade** | N1: Acesso seguro e autenticado ao sistema. |
| **Caso de Teste** | Verificar rejeição de login com senha incorreta (base para acesso de todos os atores ao sistema). |
| **Pré-condições** | Usuário cadastrado no sistema. |
| **Dados de Entrada** | E-mail válido e senha incorreta. |
| **Ações** | 1. Acessar a tela de login.<br>2. Inserir e-mail correto e senha incorreta.<br>3. Confirmar o acesso. |

| | |
|---|---|
| **Resultado Esperado** | Sistema exibe mensagem de erro informando credenciais inválidas e nega o acesso. |

Tabela 6. Teste de aceitação N1T2: Verificar rejeição de login com senha incorreta (base para todos os atores).

| | |
|---|---|
| **Identificador** | N1T3 |
| **Necessidade** | N1: Acesso seguro e autenticado ao sistema. |
| **Caso de Teste** | Verificar bloqueio de acesso a recurso restrito por perfil sem permissão, RBAC (base para todos os atores). |
| **Pré-condições** | Usuário com perfil Montador autenticado. |
| **Dados de Entrada** | - |
| **Ações** | 1. Tentar navegar até uma funcionalidade restrita ao perfil de Gestor ou RH. |
| **Resultado Esperado** | A API retorna código HTTP 403 (Forbidden) e o acesso é bloqueado. |

Tabela 7. Teste de aceitação N1T3: Verificar bloqueio de acesso a recurso restrito por perfil sem permissão, RBAC (base para todos os atores).

A necessidade N2 corresponde ao acompanhamento gerencial e de produtividade em tempo real, originada da primeira necessidade do Documento de Visão. N2T1 (Tabela 8) examina o carregamento correto do Dashboard com seus indicadores operacionais. N2T2 (Tabela 9) avalia o recebimento instantâneo de alertas de avaria via WebSocket. N2T3 (Tabela 10) cobre o fluxo de solicitação de reposição de peça à fábrica desencadeado pelo Gestor da Franquia.

| | |
|---|---|
| **Identificador** | N2T1 |
| **Necessidade** | N2: Acompanhamento gerencial e de produtividade em tempo real. |
| **Caso de Teste** | Verificar o carregamento do Dashboard com indicadores operacionais em tempo real (US08, UC11). |
| **Pré-condições** | Usuário com perfil Gestor da Franquia autenticado. |
| **Dados de Entrada** | - |
| **Ações** | 1. Acessar o Dashboard gerencial pela aplicação web. |

| | |
|---|---|
| **Resultado Esperado** | Painéis de check-ins, avarias, EPIs e produtividade são renderizados; conexão WebSocket estabelecida com sucesso. |

Tabela 8. Teste de aceitação N2T1: Verificar o carregamento do Dashboard com indicadores operacionais em tempo real (US08, UC11).

| | |
|---|---|
| **Identificador** | N2T2 |
| **Necessidade** | N2: Acompanhamento gerencial e de produtividade em tempo real. |
| **Caso de Teste** | Verificar o recebimento de alerta de avaria via WebSocket no Dashboard gerencial (US10, UC13). |
| **Pré-condições** | Gestor com o Dashboard aberto; Montador autenticado no aplicativo mobile. |
| **Dados de Entrada** | - |
| **Ações** | 1. O Montador registra uma nova avaria pelo aplicativo mobile. |
| **Resultado Esperado** | Notificação de avaria exibida no Dashboard do Gestor em menos de 3 segundos, sem necessidade de recarregar a página. |

Tabela 9. Teste de aceitação N2T2: Verificar o recebimento de alerta de avaria via WebSocket no Dashboard gerencial (US10, UC13).

| | |
|---|---|
| **Identificador** | N2T3 |
| **Necessidade** | N2: Acompanhamento gerencial e de produtividade em tempo real. |
| **Caso de Teste** | Verificar a solicitação de reposição de peça à fábrica pelo Gestor (US10, UC14). |
| **Pré-condições** | Avaria com status EM_ANALISE. |
| **Dados de Entrada** | - |
| **Ações** | 1. Selecionar a avaria na lista do Dashboard.<br>2. Acionar "Solicitar Reposição".<br>3. Confirmar. |

| | |
|---|---|
| **Resultado Esperado** | Status da avaria muda para REPOSICAO_SOLICITADA; status da peça muda para AGUARDANDO_REPOSICAO. |

Tabela 10. Teste de aceitação N2T3: Verificar a solicitação de reposição de peça à fábrica pelo Gestor (US10, UC14).

A necessidade N3 abrange o controle operacional centralizado de RH, EPIs e ferramentas, derivada da segunda necessidade do Documento de Visão. N3T1 (Tabela 11) valida o cadastro de novos montadores no sistema pelo perfil de RH. N3T2 (Tabela 12) cobre o fluxo de aprovação de atestados médicos. N3T3 (Tabela 13) testa a geração e exportação do espelho de ponto no formato AFD, conforme o padrão exigido pela Portaria 671/2021.

| | |
|---|---|
| **Identificador** | N3T1 |
| **Necessidade** | N3: Controle operacional centralizado de RH, EPIs e ferramentas. |
| **Caso de Teste** | Verificar o cadastro de novo montador no sistema (US12, UC17). |
| **Pré-condições** | Usuário com perfil Profissional de RH autenticado. |
| **Dados de Entrada** | Nome, e-mail, telefone e perfil do colaborador. |
| **Ações** | 1. Acessar "Cadastro de Montadores".<br>2. Preencher os dados do colaborador.<br>3. Salvar. |
| **Resultado Esperado** | Usuário criado com perfil MONTADOR; histórico de cadastro preservado no banco de dados. |

Tabela 11. Teste de aceitação N3T1: Verificar o cadastro de novo montador no sistema (US12, UC17).

| | |
|---|---|
| **Identificador** | N3T2 |
| **Necessidade** | N3: Controle operacional centralizado de RH, EPIs e ferramentas. |
| **Caso de Teste** | Verificar a aprovação de atestado médico submetido pelo montador (US13, UC18). |
| **Pré-condições** | Atestado com status PENDENTE_DE_VALIDACAO. |
| **Dados de Entrada** | - |

| | |
|---|---|
| **Ações** | 1. Acessar a lista de atestados pendentes.<br>2. Visualizar a foto do documento via Signed URL.<br>3. Acionar "Aprovar". |
| **Resultado Esperado** | Status do atestado muda para APROVADO; jornadas do período de validade são automaticamente abonadas no banco de horas. |

Tabela 12. Teste de aceitação N3T2: Verificar a aprovação de atestado médico submetido pelo montador (US13, UC18).

| | |
|---|---|
| **Identificador** | N3T3 |
| **Necessidade** | N3: Controle operacional centralizado de RH, EPIs e ferramentas. |
| **Caso de Teste** | Verificar a visualização do espelho de ponto e a exportação do relatório AFD (US14, UC19). |
| **Pré-condições** | Montador com jornadas registradas no mês; usuário RH autenticado. |
| **Dados de Entrada** | Período selecionado para exportação. |
| **Ações** | 1. Acessar o banco de horas do montador.<br>2. Selecionar o período desejado.<br>3. Acionar "Exportar AFD". |
| **Resultado Esperado** | Espelho de ponto exibido com todas as marcações e saldo calculado; arquivo AFD gerado conforme o padrão da Portaria 671/2021. |

Tabela 13. Teste de aceitação N3T3: Verificar a visualização do espelho de ponto e a exportação do relatório AFD (US14, UC19).

A necessidade N4 trata da garantia de assertividade no descarregamento de peças no cliente, terceira necessidade do Documento de Visão. N4T1 (Tabela 14) avalia a leitura bem-sucedida de uma peça pertencente ao pedido ativo. N4T2 (Tabela 15) verifica a geração do alerta de inconsistência quando uma peça de pedido incorreto é lida. N4T3 (Tabela 16) testa o comportamento do aplicativo em modo offline, assegurando que as validações e os alertas funcionem sem conexão de rede.

| | |
|---|---|
| **Identificador** | N4T1 |
| **Necessidade** | N4: Garantia de assertividade no descarregamento de peças no cliente. |
| **Caso de Teste** | Verificar a leitura de código de barras de peça válida pertencente ao pedido ativo (US02, UC03). |

| | |
|---|---|
| **Pré-condições** | Pedido ativo com peças triadas; aplicativo com dados do pedido sincronizados localmente. |
| **Dados de Entrada** | Código de barras de uma peça pertencente ao pedido ativo. |
| **Ações** | 1. Abrir o módulo de scanner no aplicativo.<br>2. Ler o código de barras de uma peça pertencente ao pedido ativo. |
| **Resultado Esperado** | Peça muda para status DESCARREGADA e o contador de itens conferidos é atualizado. |

Tabela 14. Teste de aceitação N4T1: Verificar a leitura de código de barras de peça válida pertencente ao pedido ativo (US02, UC03).

| | |
|---|---|
| **Identificador** | N4T2 |
| **Necessidade** | N4: Garantia de assertividade no descarregamento de peças no cliente. |
| **Caso de Teste** | Verificar o alerta de inconsistência ao ler peça de pedido incorreto (US02, UC04). |
| **Pré-condições** | Peça pertencente a um pedido diferente do pedido ativo. |
| **Dados de Entrada** | Código de barras de uma peça de outro pedido. |
| **Ações** | 1. Ler o código de barras de uma peça que pertence a outro pedido. |
| **Resultado Esperado** | Sistema exibe alerta informando que a peça não pertence ao pedido ativo e a registra com status INCONSISTENTE. |

Tabela 15. Teste de aceitação N4T2: Verificar o alerta de inconsistência ao ler peça de pedido incorreto (US02, UC04).

| | |
|---|---|
| **Identificador** | N4T3 |
| **Necessidade** | N4: Garantia de assertividade no descarregamento de peças no cliente. |
| **Caso de Teste** | Verificar a conferência de peças em modo offline com persistência local (US02, UC03). |
| **Pré-condições** | Aplicativo sem conexão com a internet; dados do pedido disponíveis no banco de dados local (WatermelonDB). |

| | |
|---|---|
| **Dados de Entrada** | Códigos de barras das peças do pedido. |
| **Ações** | 1. Desativar Wi-Fi e dados móveis.<br>2. Realizar a leitura dos códigos de barras das peças do pedido. |
| **Resultado Esperado** | Registros persistidos localmente no WatermelonDB; alertas de inconsistência funcionam sem necessidade de conexão de rede. |

Tabela 16. Teste de aceitação N4T3: Verificar a conferência de peças em modo offline com persistência local (US02, UC03).

A necessidade N5 trata da comunicação instantânea e rastreável de avarias, quarta necessidade do Documento de Visão. N5T1 (Tabela 17) valida o registro completo de uma avaria com foto e categorização pelo montador, incluindo a notificação em tempo real ao Gestor. N5T2 (Tabela 18) confirma que o sistema impede o envio de avarias sem a evidência fotográfica obrigatória. N5T3 (Tabela 19) verifica que avarias registradas em modo offline são corretamente sincronizadas ao restabelecer a conexão.

| | |
|---|---|
| **Identificador** | N5T1 |
| **Necessidade** | N5: Comunicação instantânea e rastreável de avarias. |
| **Caso de Teste** | Verificar o registro de avaria com foto e categoria pelo montador (US03, UC05). |
| **Pré-condições** | Peça previamente lida e confirmada no pedido ativo. |
| **Dados de Entrada** | Tipo de avaria selecionado e foto capturada. |
| **Ações** | 1. Selecionar a peça com dano.<br>2. Acionar "Reportar Avaria".<br>3. Selecionar o tipo de avaria.<br>4. Capturar a foto do dano.<br>5. Confirmar o envio. |
| **Resultado Esperado** | Avaria salva com status REPORTADA e foto armazenada no Supabase Storage; peça muda para COM_AVARIA; Gestor recebe notificação via WebSocket em menos de 3 segundos. |

Tabela 17. Teste de aceitação N5T1: Verificar o registro de avaria com foto e categoria pelo montador (US03, UC05).

| | |
|---|---|
| **Identificador** | N5T2 |
| **Necessidade** | N5: Comunicação instantânea e rastreável de avarias. |
| **Caso de Teste** | Verificar o bloqueio de envio de avaria sem foto obrigatória (US03, UC05). |
| **Pré-condições** | Tela de registro de avaria aberta. |
| **Dados de Entrada** | Tipo de avaria selecionado, sem foto capturada. |
| **Ações** | 1. Preencher o tipo de avaria sem capturar foto.<br>2. Tentar confirmar o envio. |
| **Resultado Esperado** | Sistema bloqueia o envio e exibe mensagem indicando que a foto é obrigatória para o registro da avaria. |

Tabela 18. Teste de aceitação N5T2: Verificar o bloqueio de envio de avaria sem foto obrigatória (US03, UC05).

| | |
|---|---|
| **Identificador** | N5T3 |
| **Necessidade** | N5: Comunicação instantânea e rastreável de avarias. |
| **Caso de Teste** | Verificar o registro de avaria em modo offline com sincronização posterior (US03, UC05). |
| **Pré-condições** | Aplicativo sem conexão com a internet. |
| **Dados de Entrada** | Tipo de avaria selecionado e foto capturada. |
| **Ações** | 1. Registrar avaria com foto em modo offline. |
| **Resultado Esperado** | Avaria e foto salvas localmente no WatermelonDB; sincronização com a API ocorre automaticamente ao restabelecer a conexão. |

Tabela 19. Teste de aceitação N5T3: Verificar o registro de avaria em modo offline com sincronização posterior (US03, UC05).

A necessidade N6 contempla o registro e a validação da jornada de trabalho com geolocalização, expandida a partir da segunda necessidade do Documento de Visão para detalhar os requisitos de conformidade trabalhista. N6T1 (Tabela 20) testa o registro de check-in com coordenadas GPS dentro do raio de tolerância configurado. N6T2 (Tabela 21) verifica o bloqueio da marcação quando o montador está fora da área permitida. N6T3 (Tabela 22) valida o ciclo completo das quatro marcações diárias exigidas pela CLT.

| | |
|---|---|
| **Identificador** | N6T1 |
| **Necessidade** | N6: Registro e validação da jornada de trabalho com geolocalização. |
| **Caso de Teste** | Verificar o registro de check-in com geolocalização válida (US01, UC01, UC02). |
| **Pré-condições** | Cliente cadastrado com endereço georreferenciado; montador posicionado a menos de 200 m do local de serviço. |
| **Dados de Entrada** | - |
| **Ações** | 1. Abrir o aplicativo.<br>2. Selecionar o pedido ativo.<br>3. Acionar "Iniciar Jornada" e conceder permissão de GPS. |
| **Resultado Esperado** | Marcação de entrada registrada com coordenadas válidas; jornada com status EM_ANDAMENTO. |

Tabela 20. Teste de aceitação N6T1: Verificar o registro de check-in com geolocalização válida (US01, UC01, UC02).

| | |
|---|---|
| **Identificador** | N6T2 |
| **Necessidade** | N6: Registro e validação da jornada de trabalho com geolocalização. |
| **Caso de Teste** | Verificar o bloqueio de check-in fora do raio de tolerância permitido (US01, UC02). |
| **Pré-condições** | Montador posicionado a mais de 200 m do endereço do cliente. |
| **Dados de Entrada** | - |
| **Ações** | 1. Acionar "Iniciar Jornada" estando fora do raio de tolerância do local de serviço. |
| **Resultado Esperado** | Sistema exibe alerta informando que o montador está fora da área permitida e impede o registro da marcação. |

Tabela 21. Teste de aceitação N6T2: Verificar o bloqueio de check-in fora do raio de tolerância permitido (US01, UC02).

| | |
|---|---|
| **Identificador** | N6T3 |
| **Necessidade** | N6: Registro e validação da jornada de trabalho com geolocalização. |
| **Caso de Teste** | Verificar o registro das quatro marcações diárias previstas em CLT (US01, UC01). |
| **Pré-condições** | Jornada iniciada com status EM_ANDAMENTO. |
| **Dados de Entrada** | - |
| **Ações** | 1. Acionar "Pausa, Almoço".<br>2. Acionar "Retornar".<br>3. Acionar "Encerrar Jornada". |
| **Resultado Esperado** | As quatro marcações (entrada, saída para almoço, retorno e saída final) são registradas; status da jornada muda para FINALIZADA. |

Tabela 22. Teste de aceitação N6T3: Verificar o registro das quatro marcações diárias previstas em CLT (US01, UC01).

A necessidade N7 cobre a triagem e a rastreabilidade de peças no depósito, expandida a partir da terceira necessidade do Documento de Visão para detalhar o fluxo logístico de recebimento. N7T1 (Tabela 23) avalia o cadastro de peça via leitura de código de barras e o agrupamento automático por pedido e ambiente. N7T2 (Tabela 24) verifica se o sistema impede o registro duplicado de um mesmo código de barras. N7T3 (Tabela 25) testa a exibição agrupada das peças triadas, permitindo a conferência visual antes do despacho.

| | |
|---|---|
| **Identificador** | N7T1 |
| **Necessidade** | N7: Triagem e rastreabilidade de peças no depósito. |
| **Caso de Teste** | Verificar o cadastro de peça via leitura de código de barras e o agrupamento por pedido e ambiente (US07, UC09, UC10). |
| **Pré-condições** | Pedido cadastrado; peça com status AGUARDANDO_TRIAGEM. |
| **Dados de Entrada** | Código de barras da peça recebida da fábrica. |
| **Ações** | 1. Ler o código de barras da peça recebida da fábrica.<br>2. Confirmar o pedido e o ambiente de destino.<br>3. Salvar. |

| | |
|---|---|
| **Resultado Esperado** | Peça muda para status TRIADA; o sistema agrupa o item automaticamente por pedido e ambiente de montagem. |

Tabela 23. Teste de aceitação N7T1: Verificar o cadastro de peça via leitura de código de barras e o agrupamento por pedido e ambiente (US07, UC09, UC10).

| | |
|---|---|
| **Identificador** | N7T2 |
| **Necessidade** | N7: Triagem e rastreabilidade de peças no depósito. |
| **Caso de Teste** | Verificar o bloqueio de cadastro de código de barras duplicado (US07, UC09). |
| **Pré-condições** | Peça com o mesmo código de barras já triada no sistema. |
| **Dados de Entrada** | Código de barras de peça já cadastrada. |
| **Ações** | 1. Tentar ler o código de barras de uma peça já cadastrada. |
| **Resultado Esperado** | Sistema exibe alerta informando que a peça já foi cadastrada e impede o registro duplicado. |

Tabela 24. Teste de aceitação N7T2: Verificar o bloqueio de cadastro de código de barras duplicado (US07, UC09).

| | |
|---|---|
| **Identificador** | N7T3 |
| **Necessidade** | N7: Triagem e rastreabilidade de peças no depósito. |
| **Caso de Teste** | Verificar a visualização de peças agrupadas por pedido e ambiente após a triagem (US07, UC10). |
| **Pré-condições** | Pedido com múltiplas peças triadas em ambientes distintos (por exemplo, Cozinha e Quarto). |
| **Dados de Entrada** | - |
| **Ações** | 1. Acessar o pedido na interface web.<br>2. Visualizar a listagem de peças do pedido. |

| | |
|---|---|
| **Resultado Esperado** | O sistema exibe as peças organizadas por ambiente de montagem, permitindo ao Operador Logístico identificar visualmente o agrupamento correto antes do despacho. |

Tabela 25. Teste de aceitação N7T3: Verificar a visualização de peças agrupadas por pedido e ambiente após a triagem (US07, UC10).

A necessidade N8 trata do envio e da proteção de atestados médicos, com foco na conformidade com a Lei Geral de Proteção de Dados (LGPD). É uma expansão da segunda necessidade do Documento de Visão, motivada pela classificação dos atestados como dados sensíveis. N8T1 (Tabela 26) valida o fluxo de envio do documento pelo aplicativo e o armazenamento seguro em bucket privado. N8T2 (Tabela 27) garante que o acesso ao módulo de atestados é bloqueado para perfis sem permissão. N8T3 (Tabela 28) cobre o fluxo de rejeição pelo RH, com registro obrigatório do motivo e notificação ao montador.

| | |
|---|---|
| **Identificador** | N8T1 |
| **Necessidade** | N8: Envio e proteção de atestados médicos. |
| **Caso de Teste** | Verificar o envio de atestado médico pelo aplicativo (US04, UC06). |
| **Pré-condições** | Usuário Montador autenticado. |
| **Dados de Entrada** | Foto do documento médico, data de início e data de fim do período de validade. |
| **Ações** | 1. Acessar "Enviar Atestado".<br>2. Capturar foto do documento médico.<br>3. Informar o período de validade.<br>4. Enviar. |
| **Resultado Esperado** | Atestado salvo com status PENDENTE_DE_VALIDACAO; foto armazenada em bucket privado no Supabase Storage com acesso protegido. |

Tabela 26. Teste de aceitação N8T1: Verificar o envio de atestado médico pelo aplicativo (US04, UC06).

| | |
|---|---|
| **Identificador** | N8T2 |
| **Necessidade** | N8: Envio e proteção de atestados médicos. |
| **Caso de Teste** | Verificar o bloqueio de acesso ao módulo de atestados por perfil sem permissão, conformidade com a LGPD (US04, UC06). |

| | |
|---|---|
| **Pré-condições** | Usuário com perfil Gestor autenticado. |
| **Dados de Entrada** | - |
| **Ações** | 1. Tentar acessar o módulo de atestados médicos pela interface web. |
| **Resultado Esperado** | Sistema retorna HTTP 403 (Forbidden); a foto do atestado não é acessível fora do perfil de RH, em conformidade com a Lei Geral de Proteção de Dados (LGPD). |

Tabela 27. Teste de aceitação N8T2: Verificar o bloqueio de acesso ao módulo de atestados por perfil sem permissão, conformidade com a LGPD (US04, UC06).

| | |
|---|---|
| **Identificador** | N8T3 |
| **Necessidade** | N8: Envio e proteção de atestados médicos. |
| **Caso de Teste** | Verificar a rejeição de atestado médico com preenchimento de motivo pelo RH (US13, UC18). |
| **Pré-condições** | Atestado com status PENDENTE_DE_VALIDACAO; usuário com perfil RH autenticado. |
| **Dados de Entrada** | Motivo da rejeição preenchido pelo profissional de RH. |
| **Ações** | 1. Acessar a lista de atestados pendentes.<br>2. Visualizar a foto do documento via Signed URL.<br>3. Acionar "Rejeitar".<br>4. Preencher o motivo da rejeição.<br>5. Confirmar. |
| **Resultado Esperado** | Status do atestado muda para REJEITADO; motivo da rejeição registrado no banco de dados; notificação enviada ao montador informando a reprovação do documento. |

Tabela 28. Teste de aceitação N8T3: Verificar a rejeição de atestado médico com preenchimento de motivo pelo RH (US13, UC18).

## 6.2 Testes de Integração

Os testes de integração têm como escopo os pontos de contato entre os subsistemas da arquitetura do MontaControl. Enquanto os testes de aceitação validam o comportamento funcional do sistema do ponto de vista do usuário, os testes de integração verificam se o fluxo de dados percorre

corretamente os subsistemas envolvidos em cada operação, como o aplicativo mobile, a API REST, o banco de dados PostgreSQL, o Supabase Storage, o Servidor WebSocket e a API de Geolocalização (Nominatim/OpenStreetMap). Além das colunas presentes nos testes de aceitação, cada tabela desta seção inclui os campos sistemas envolvidos e interface de comunicação, que identificam quais subsistemas participam do teste e qual protocolo os interliga. Os dez testes de integração estão distribuídos nas Tabelas 29 a 38.

| | |
|---|---|
| **Identificador** | TI01 |
| **Caso de Teste** | Verificar a autenticação do montador no aplicativo e a obtenção de token JWT. |
| **Sistemas Envolvidos** | Aplicativo Mobile e API REST. |
| **Interface** | Requisição HTTP no formato REST. |
| **Pré-condições** | Usuário Montador cadastrado e ativo no sistema. |
| **Dados de Entrada** | E-mail e senha válidos. |
| **Ações** | 1. Inserir credenciais válidas na tela de login do aplicativo.<br>2. Confirmar o acesso. |
| **Resultado Esperado** | A API retorna token JWT (JSON Web Token); o aplicativo armazena o token e redireciona o usuário para a tela inicial do perfil Montador. |

Tabela 29. Teste de integração TI01: Verificar a autenticação do montador no aplicativo e a obtenção de token JWT.

| | |
|---|---|
| **Identificador** | TI02 |
| **Caso de Teste** | Verificar a rejeição de requisição com token JWT expirado. |
| **Sistemas Envolvidos** | Aplicativo Mobile e API REST. |
| **Interface** | Requisição HTTP no formato REST. |
| **Pré-condições** | Usuário autenticado com token cujo prazo de validade expirou. |
| **Dados de Entrada** | Token JWT expirado. |
| **Ações** | 1. Realizar qualquer requisição autenticada após a expiração do token. |

| | |
|---|---|
| **Resultado Esperado** | A API retorna HTTP 401 (Unauthorized); o aplicativo solicita ao usuário que realize novo login. |

Tabela 30. Teste de integração TI02: Verificar a rejeição de requisição com token JWT expirado.

| | |
|---|---|
| **Identificador** | TI03 |
| **Caso de Teste** | Verificar o registro de check-in com validação de geolocalização via API (UC01, UC02). |
| **Sistemas Envolvidos** | Aplicativo Mobile, API REST e API de Geolocalização (Nominatim/OpenStreetMap). |
| **Interface** | Requisição HTTP no formato REST. |
| **Pré-condições** | Montador autenticado e posicionado a menos de 200 m do endereço do cliente. |
| **Dados de Entrada** | Coordenadas GPS capturadas pelo dispositivo. |
| **Ações** | 1. Acionar "Iniciar Jornada" no aplicativo com GPS ativo.<br>2. Aguardar a resposta da API. |
| **Resultado Esperado** | A API de Geolocalização confirma a proximidade com o endereço do cliente; marcação de entrada persistida no PostgreSQL com checkin_valido = true. |

Tabela 31. Teste de integração TI03: Verificar o registro de check-in com validação de geolocalização via API (UC01, UC02).

| | |
|---|---|
| **Identificador** | TI04 |
| **Caso de Teste** | Verificar a persistência local da jornada em modo offline e a sincronização posterior (UC01). |
| **Sistemas Envolvidos** | Aplicativo Mobile (WatermelonDB) e API REST. |
| **Interface** | Requisição HTTP no formato REST (após restauração da conexão). |
| **Pré-condições** | Dispositivo sem conexão com a internet. |
| **Dados de Entrada** | Marcações de jornada realizadas em modo offline. |

| | |
|---|---|
| **Ações** | 1. Realizar todas as marcações de jornada com Wi-Fi e dados móveis desativados.<br>2. Reativar a conexão com a internet. |
| **Resultado Esperado** | Marcações persistidas localmente no WatermelonDB; ao restabelecer a conexão, o Sync Manager envia os dados via REST e a API persiste os registros no PostgreSQL. |

Tabela 32. Teste de integração TI04: Verificar a persistência local da jornada em modo offline e a sincronização posterior (UC01).

| | |
|---|---|
| **Identificador** | TI05 |
| **Caso de Teste** | Verificar a sincronização de leituras de código de barras realizadas offline (UC03). |
| **Sistemas Envolvidos** | Aplicativo Mobile (WatermelonDB) e API REST. |
| **Interface** | Requisição HTTP no formato REST (após restauração da conexão). |
| **Pré-condições** | Aplicativo sem conexão com a internet; dados do pedido disponíveis localmente. |
| **Dados de Entrada** | Códigos de barras lidos em modo offline. |
| **Ações** | 1. Realizar a leitura dos códigos de barras das peças em modo offline.<br>2. Restaurar a conexão com a internet. |
| **Resultado Esperado** | Leituras salvas localmente com flag de pendência; após a sincronização, a API atualiza o status das peças no PostgreSQL para DESCARREGADA ou INCONSISTENTE conforme a regra de negócio. |

Tabela 33. Teste de integração TI05: Verificar a sincronização de leituras de código de barras realizadas offline (UC03).

| | |
|---|---|
| **Identificador** | TI06 |
| **Caso de Teste** | Verificar o upload de foto de avaria para o Supabase Storage e a notificação ao Gestor via WebSocket (UC05). |
| **Sistemas Envolvidos** | Aplicativo Mobile, API REST, Supabase Storage e Servidor WebSocket. |

| | |
|---|---|
| **Interface** | Requisição HTTP no formato REST e protocolo WebSocket seguro (WSS). |
| **Pré-condições** | Peça previamente lida; Gestor com Dashboard aberto. |
| **Dados de Entrada** | Tipo de avaria e foto capturada. |
| **Ações** | 1. Registrar avaria e capturar foto pelo aplicativo.<br>2. Confirmar o envio. |
| **Resultado Esperado** | Foto armazenada no Supabase Storage (bucket privado); registro de avaria salvo no PostgreSQL; evento emitido via WebSocket; notificação exibida no Dashboard do Gestor em menos de 3 segundos. |

Tabela 34. Teste de integração TI06: Verificar o upload de foto de avaria para o Supabase Storage e a notificação ao Gestor via WebSocket (UC05).

| | |
|---|---|
| **Identificador** | TI07 |
| **Caso de Teste** | Verificar o upload de foto de atestado médico e a geração de Signed URL com controle de acesso LGPD (UC06). |
| **Sistemas Envolvidos** | Aplicativo Mobile, API REST e Supabase Storage. |
| **Interface** | Requisição HTTP no formato REST. |
| **Pré-condições** | Usuário Montador autenticado. |
| **Dados de Entrada** | Foto do documento médico, data de início e data de fim do período. |
| **Ações** | 1. Acessar "Enviar Atestado".<br>2. Capturar foto do documento médico.<br>3. Informar o período de validade e enviar. |
| **Resultado Esperado** | Foto armazenada em bucket privado no Supabase Storage; atestado salvo com status PENDENTE_DE_VALIDACAO; acesso ao arquivo disponível exclusivamente para o perfil RH por meio de Signed URL com expiração temporal, em conformidade com a LGPD. |

Tabela 35. Teste de integração TI07: Verificar o upload de foto de atestado médico e a geração de Signed URL com controle de acesso LGPD (UC06).

| | |
|---|---|
| **Identificador** | TI08 |

| | |
|---|---|
| **Caso de Teste** | Verificar a triagem de peça via código de barras e o agrupamento automático por pedido e ambiente (UC09, UC10). |
| **Sistemas Envolvidos** | Aplicação Web e API REST. |
| **Interface** | Requisição HTTP no formato REST. |
| **Pré-condições** | Pedido cadastrado; peça com status AGUARDANDO_TRIAGEM. |
| **Dados de Entrada** | Código de barras da peça, pedido e ambiente de destino. |
| **Ações** | 1. Ler o código de barras da peça na interface web.<br>2. Confirmar pedido e ambiente de destino.<br>3. Salvar. |
| **Resultado Esperado** | A API processa a regra de agrupamento; peça persiste com status TRIADA e associada ao pedido e ambiente corretos no PostgreSQL; interface exibe confirmação visual. |

Tabela 36. Teste de integração TI08: Verificar a triagem de peça via código de barras e o agrupamento automático por pedido e ambiente (UC09, UC10).

| | |
|---|---|
| **Identificador** | TI09 |
| **Caso de Teste** | Verificar a validação de atestado médico pelo RH com acesso ao arquivo via Signed URL (UC18). |
| **Sistemas Envolvidos** | Aplicação Web, API REST e Supabase Storage. |
| **Interface** | Requisição HTTP no formato REST. |
| **Pré-condições** | Atestado com status PENDENTE_DE_VALIDACAO; usuário com perfil RH autenticado. |
| **Dados de Entrada** | - |
| **Ações** | 1. Acessar a lista de atestados pendentes.<br>2. Clicar na foto (Signed URL) para visualizar o documento.<br>3. Acionar "Aprovar" ou "Rejeitar" com justificativa. |

| | |
|---|---|
| **Resultado Esperado** | A API realiza consulta cruzada entre PostgreSQL e Supabase Storage; status do atestado é atualizado; jornadas do período são abonadas (aprovação) ou notificação é enviada ao montador (rejeição). |

Tabela 37. Teste de integração TI09: Verificar a validação de atestado médico pelo RH com acesso ao arquivo via Signed URL (UC18).

| | |
|---|---|
| **Identificador** | TI10 |
| **Caso de Teste** | Verificar a exportação do relatório AFD pelo setor de RH (UC19). |
| **Sistemas Envolvidos** | Aplicação Web e API REST. |
| **Interface** | Requisição HTTP no formato REST. |
| **Pré-condições** | Montador com jornadas registradas no período selecionado; usuário RH autenticado. |
| **Dados de Entrada** | Período selecionado para exportação. |
| **Ações** | 1. Acessar o módulo de banco de horas.<br>2. Selecionar o período desejado.<br>3. Acionar "Exportar AFD". |
| **Resultado Esperado** | A API consolida os logs de check-in, check-out e pausas; arquivo AFD gerado conforme o padrão da Portaria 671/2021 e disponibilizado para download na interface web. |

Tabela 38. Teste de integração TI10: Verificar a exportação do relatório AFD pelo setor de RH (UC19).

# 7. Cronograma e Processo de Implementação

Nesta seção são apresentados o cronograma de desenvolvimento do sistema MontaControl e o processo de implementação do mesmo. A Seção 7.1 descreve o cronograma geral, detalhando a divisão das atividades em oito sprints quinzenais, com seus respectivos períodos e responsáveis. A Seção 7.2 descreve as práticas, ferramentas e artefatos que guiarão o processo de implementação ao longo do desenvolvimento.

## 7.1 Cronograma

O cronograma de desenvolvimento do MontaControl está dividido em duas etapas principais. A primeira etapa, correspondente ao TCC 1, compreende a elaboração da documentação do projeto, abrangendo o período de fevereiro de 2026 a junho de 2026. A segunda etapa, referente ao TCC 2,

corresponde à implementação efetiva do sistema, com início previsto para 01 de agosto de 2026 e conclusão em 12 de dezembro de 2026.

O desenvolvimento do sistema foi organizado em oito sprints de quinze dias cada, conforme apresentado na Tabela 23. As atividades de cada sprint foram distribuídas entre os autores David Dias Pinto, responsável pela implementação do *frontend mobile* e das interfaces *web*, e Vinicius Salles de Oliveira, responsável pelo *backend*, banco de dados e infraestrutura. Essa divisão foi estabelecida com base nos subsistemas identificados durante a documentação do projeto.

| **Nome** | **Período** | **David Dias Pinto** | **Vinicius Salles de Oliveira** |
|---|---|---|---|
| | | **Atividades** | **Atividades** |
| Sprint 1 | 01/08/2026 - 15/08/2026 | ● **Fundação do projeto Mobile**<br>○ Configuração do repositório monorepo e estrutura de pastas<br>○ Configuração do projeto React Native com TypeScript<br>○ Definição do tema visual (cores, tipografia, espaçamentos)<br>○ Implementação dos componentes base reutilizáveis (botões, inputs, cards, modais)<br>○ Configuração do WatermelonDB para persistência local offline-first<br>○ Implementação das telas de Login e Onboarding do aplicativo | ● **Fundação do projeto Backend**<br>○ Configuração do projeto NestJS com TypeScript e estrutura modular<br>○ Configuração do Prisma ORM e conexão com Supabase/PostgreSQL<br>○ Implementação das migrações iniciais do banco de dados<br>○ Implementação do módulo de autenticação com JWT<br>○ Implementação do controle de acesso por perfil (RBAC) com Guards<br>○ Implementação das rotas de autenticação e recuperação de senha<br>○ Implementação de testes de integração do módulo de autenticação |
| Sprint 2 | 16/08/2026 - 30/08/2026 | ● **Módulo de Jornada de Trabalho (Mobile)** | ● **Módulo de Gestão de Pessoal (Backend)** |

| **Nome** | **Período** | **David Dias Pinto** | **Vinicius Salles de Oliveira** |
|---|---|---|---|
| | | **Atividades** | **Atividades** |
| | | ○ Implementação do fluxo de check-in com captura de geolocalização (GPS)<br>○ Implementação do fluxo de check-out e registro de pausas<br>○ Persistência offline das marcações via WatermelonDB<br>○ Implementação da tela de consulta de banco de horas<br>○ Testes de aceitação do fluxo de jornada (US01, US05) | ○ Implementação das tabelas de usuários, jornadas e pausas no Prisma<br>○ Implementação das rotas de check-in e check-out com validação de geolocalização<br>○ Implementação da regra de distância tolerada via API de Geolocalização<br>○ Implementação do cálculo de banco de horas<br>○ Implementação do endpoint de espelho de ponto<br>○ Testes de integração do módulo de jornada |
| Sprint 3 | 31/08/2026 - 14/09/2026 | ● **Módulo de Conferência de Peças (Mobile)**<br>○ Implementação do leitor de código de barras no aplicativo<br>○ Lógica de validação offline das peças contra o pedido local<br>○ Implementação de alertas de inconsistência (peça errada, faltante)<br>○ Implementação do Sync Manager para sincronização ao restaurar conexão<br>○ Testes de aceitação do fluxo de | ● **Módulo de Logística e Triagem (Backend + Web)**<br>○ Implementação das tabelas de clientes, pedidos, ambientes e peças no Prisma<br>○ Implementação das rotas de cadastro e triagem de peças via código de barras<br>○ Implementação da lógica de agrupamento por pedido e ambiente<br>○ Implementação da interface Web do Operador Logístico<br>○ Testes de integração do módulo logístico |

| Nome | Período | David Dias Pinto | Vinicius Salles de Oliveira |
|---|---|---|---|
| | | **Atividades** | **Atividades** |
| | | conferência offline (US02) | |
| Sprint 4 | 15/09/2026 - 29/09/2026 | ● **Módulo de Avarias (Mobile)**<br>○ Implementação da tela de registro de avaria com captura fotográfica<br>○ Integração com Supabase Storage para upload de imagens<br>○ Implementação da categorização por tipo de avaria<br>○ Implementação do fluxo de reenvio offline de avarias pendentes<br>○ Testes de aceitação do fluxo de registro de avaria (US03) | ● **Módulo de Avarias (Backend) e WebSocket**<br>○ Implementação da tabela de avarias e fotos no Prisma<br>○ Implementação das rotas de criação e consulta de avarias<br>○ Configuração do servidor Socket.io e implementação do serviço de notificações<br>○ Disparo de evento WebSocket ao receber nova avaria<br>○ Implementação das rotas de solicitação de reposição à fábrica<br>○ Testes de integração do módulo de avarias |
| Sprint 5 | 30/09/2026 - 14/10/2026 | ● **Módulo de Atestados e Normas (Mobile)**<br>○ Implementação da tela de envio de atestados médicos com captura de foto<br>○ Upload seguro de imagens para o Supabase Storage<br>○ Implementação da tela de consulta às normas operacionais<br>○ Testes de aceitação dos fluxos de | ● **Módulo de RH e Atestados (Backend + Web)**<br>○ Implementação das tabelas de atestados com soft delete no Prisma<br>○ Implementação das rotas de envio, validação e rejeição de atestados<br>○ Geração de Signed URLs com expiração para |

| **Nome** | **Período** | **David Dias Pinto** | **Vinicius Salles de Oliveira** |
|---|---|---|---|
| | | **Atividades** | **Atividades** |
| | | atestado e normas (US04, US06) | acesso controlado (LGPD)<br>○ Implementação das interfaces *Web* do RH (Atestados, Banco de Horas)<br>○ Implementação da exportação do relatório AFD (Portaria 671/2021)<br>○ Testes de integração e de conformidade LGPD |
| Sprint 6 | 15/10/2026 - 29/10/2026 | ● **Dashboard Gerencial (Web - Gestor)**<br>○ Implementação da conexão WebSocket no frontend (Socket.io Client)<br>○ Implementação do Dashboard com alertas em tempo real<br>○ Implementação dos gráficos de produtividade (meta vs. realizado)<br>○ Implementação da visualização e tratamento de avarias pelo Gestor<br>○ Testes de aceitação do Dashboard em tempo real (US08, US09, US10) | ● **Módulo de Recursos (Backend + Web)**<br>○ Implementação das tabelas de ferramentas, cautelas e EPIs no Prisma<br>○ Implementação das rotas de cautela de ferramentas (empréstimo e devolução)<br>○ Implementação da lógica de detecção de cautelas atrasadas<br>○ Implementação das rotas de EPIs com alerta de vencimento<br>○ Implementação da interface Web de gestão de recursos pelo Gestor<br>○ Testes de integração do módulo de recursos |

| **Nome** | **Período** | **David Dias Pinto** | **Vinicius Salles de Oliveira** |
|---|---|---|---|
| | | **Atividades** | **Atividades** |
| Sprint 7 | 30/10/2026 - 13/11/2026 | ● **Módulo de Administração e Cadastros (Web - RH)**<br>○ Implementação da tela de cadastro e inativação de montadores<br>○ Implementação da visualização do espelho de ponto completo<br>○ Implementação do painel de gestão de banco de horas com aprovação de compensações<br>○ Testes de aceitação dos fluxos de RH (US12, US13, US14) | ● **CI/CD e Qualidade**<br>○ Configuração do pipeline de integração contínua via GitHub Actions<br>○ Automação da execução de testes de integração do backend a cada pull request<br>○ Configuração do ambiente de homologação no Supabase<br>○ Revisão geral de segurança e validação de conformidade LGPD |
| Sprint 8 | 14/11/2026 - 28/11/2026 | ● **Implantação e Encerramento**<br>○ Implantação da API e banco de dados em ambiente de produção<br>○ Publicação do aplicativo mobile em loja para testes finais<br>○ Documentação do guia de instalação e utilização do sistema<br>○ Redação do Post-Mortem do projeto | ● **Implantação e Encerramento**<br>○ Implantação da aplicação web em ambiente de produção<br>○ Validação final dos fluxos integrados em ambiente produtivo<br>○ Documentação do guia de instalação e utilização do sistema<br>○ Redação do Post-Mortem do projeto |

Tabela 39. Cronograma de Atividades de Desenvolvimento do Sistema MontaControl.

## 7.2 Processo de Implementação

Esta seção descreve as práticas e ferramentas que serão adotadas durante o processo de implementação do MontaControl, com o objetivo de garantir organização, rastreabilidade e qualidade ao longo de todo o desenvolvimento.

O projeto adota uma abordagem iterativa e incremental baseada no *framework* Scrum adaptado ao contexto acadêmico. As entregas são organizadas em ciclos quinzenais (*sprints*), o que permite validações frequentes das funcionalidades implementadas e a identificação precoce de eventuais desvios em relação aos requisitos levantados durante o TCC 1.

Para o controle de versões, é utilizado o Git em conjunto com o GitHub como plataforma de hospedagem do código-fonte. A organização adota um fluxo baseado em branches, onde cada nova funcionalidade é desenvolvida em uma branch dedicada. Ao término da implementação, é aberto um pull request para revisão do código, e somente após a aprovação o conteúdo é integrado à branch principal. Para garantir a consistência nesse processo, é configurado um pipeline de integração contínua por meio do GitHub Actions, que executa automaticamente os testes de integração do backend a cada pull request, assegurando que nenhuma regressão seja introduzida no código validado.

O gerenciamento das tarefas é realizado por meio do sistema Kanban disponibilizado pelo GitHub Projects. As tarefas são registradas como issues e organizadas em quatro colunas: Global Backlog, contendo todas as tarefas planejadas para o projeto; Sprint Backlog, com as tarefas selecionadas para a sprint em andamento; Em Progresso, refletindo o trabalho em execução; e Concluído, agrupando as entregas finalizadas e validadas. Cada issue é vinculada à milestone correspondente à sua sprint, garantindo rastreabilidade completa entre o planejamento e a execução.

O processo de qualidade prevê, para cada sprint, a execução de testes de aceitação baseados nos casos de teste definidos na Seção 6 deste documento, verificando o comportamento funcional das histórias de usuário implementadas. Os testes de integração do backend são automatizados no pipeline de CI, enquanto os testes de aceitação do aplicativo *mobile* e da interface *web* são conduzidos manualmente pelos desenvolvedores ao final de cada ciclo. Situações críticas, como a sincronização *offline-first* e a conformidade com a LGPD no acesso a dados sensíveis, recebem atenção especial durante as revisões de cada sprint.

# Referências

INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA (IBGE). **Pesquisa Mensal de Comércio (PMC)**. Rio de Janeiro: IBGE, 2025. Disponível em: https://emobile.com.br/site/varejo/varejo-de-moveis-cresce-59-em-2024. Acesso em: 27 jun. 2026.